

居丽叶的简历项目7：医疗助手代码解读

面试准备文档：[📖 居丽叶的简历项目7：医疗助手](#)

算法部分

代码运行

- 计算资源：默认参数配置需要2张 40G的显卡，没有充足计算资源的同学可以降低batch-size，换更小的模型，将显卡数设置为1。

1. 创建虚拟环境，同时用于训练和评测。

代码块

```
1  conda create -n medix python=3.12 -y
2  conda activate medix
```

2. 安装训练依赖

代码块

```
1  cd MediX-R1/training
2  # flash-attn 必须先单独装 (构建时依赖 torch)
3  pip install flash-attn --no-build-isolation
4  pip install -e .
```

3. 配置 API 模型

训练时的 reward 打分和评测时的 judge 打分都调用同一个 API 模型，编辑 `config.py`：

代码块

```
1  LLM_CONFIG = {
2      "api_key": "<your-api-key>",
3      "model_name": "<model-name>",
4      "base_url": "<api-base-url>",
5      "temperature": 0.7,
6      "max_tokens": 8192,
7  }
```

4. 下载训练数据

代码块

```
1 cd ../
2 python -c "
3 from huggingface_hub import snapshot_download
4 snapshot_download(repo_id='MBZUAI/medix-rl-data', repo_type='dataset',
5                  local_dir='data/medix-rl-data')
```

下载评测数据：

代码块

```
1 cd eval2/
2 python -c "
3 from huggingface_hub import snapshot_download
4 snapshot_download('flaviagiammarino/vqa-rad', repo_type='dataset',
5                  local_dir='datasets_local/vqa-rad')
```

5. 训练，以 2B DAPO 为例

代码块

```
1 cd ../training
2 bash examples/medix-rl_2b_dapo.sh
```

训练完成后 checkpoint 保存在：

代码块

```
1 training/checkpoints/medix-rl_2b_dapo/global_step_XXX/actor/
```

6. 合并模型权重

训练产出的是 FSDP 分片格式，需要合并为 HuggingFace 格式后才能用 vLLM 推理。这会将 `training/checkpoints/medix-rl\_2b\_dapo/global\_step\_600/actor/` 下的分片权重合并，输出 `.safetensors` 文件到 `actor/huggingface/` 目录。

代码块

```
1 cd ../eval2
```

```
2 bash merge.sh
```

7. 评测

评测 checkpoint 在 `rad_vqa` 任务上的表现。

代码块

```
1 cd ../eval2
2 bash eval.sh
```

`eval.sh` 会依次执行三个阶段：

1. **生成回答** — 启动 vLLM server 加载合并后的 checkpoint，对 `rad_vqa` 测试集 (451 条) 生成回答
2. **API 评分** — 调用 `config.py` 中的 API 模型，逐条判断回答是否正确 (0/1)
3. **汇总分数** — 统计正确率，打印结果表格

输出文件：

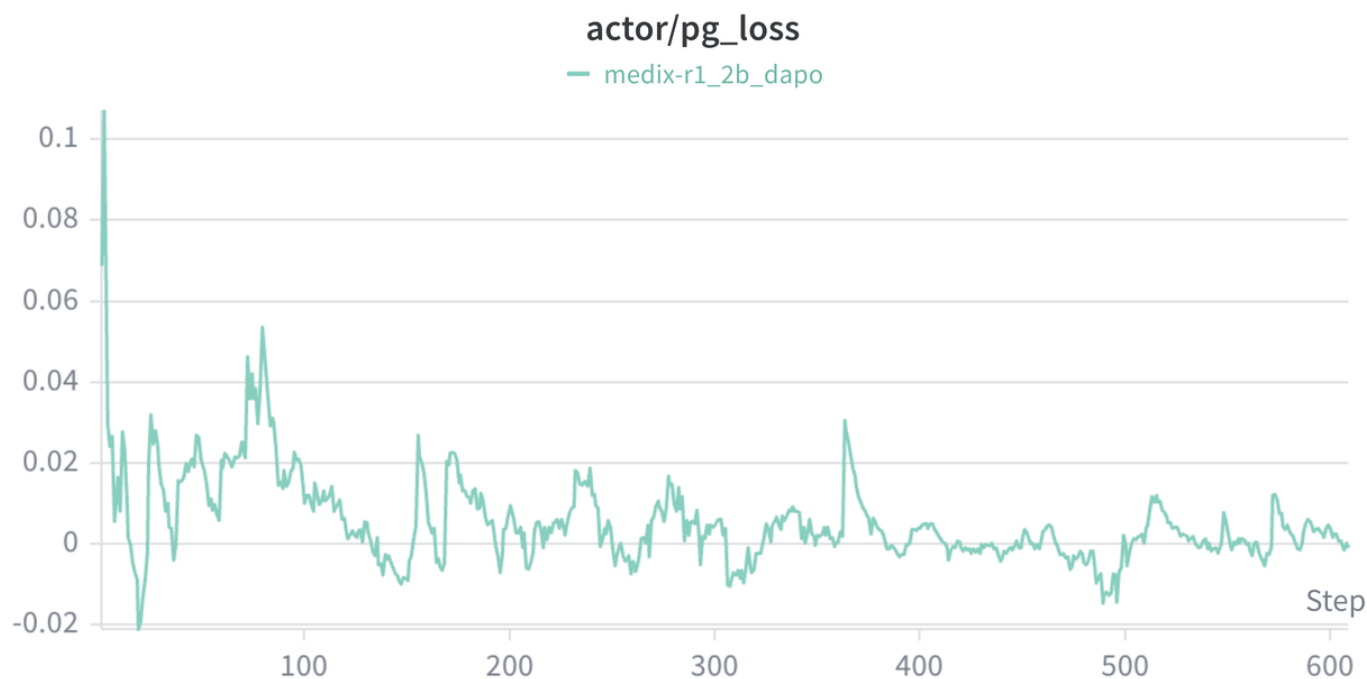
代码块

```
1 eval2/results/huggingface/
2 |— huggingface.jsonl          # 模型生成的回答
3 |— huggingface_eval.jsonl    # 每条的评分结果
4 |— huggingface_score.txt     # 最终正确率
```

训练指标分析

- 考虑到有些同学没有跑整个项目的代码，为了面试中显得自己有 RL 的经验，这里对关键训练指标进行罗列，需要能说清楚训练过程中指标的变化情况。

1. 策略梯度损失 (actor/pg_loss)

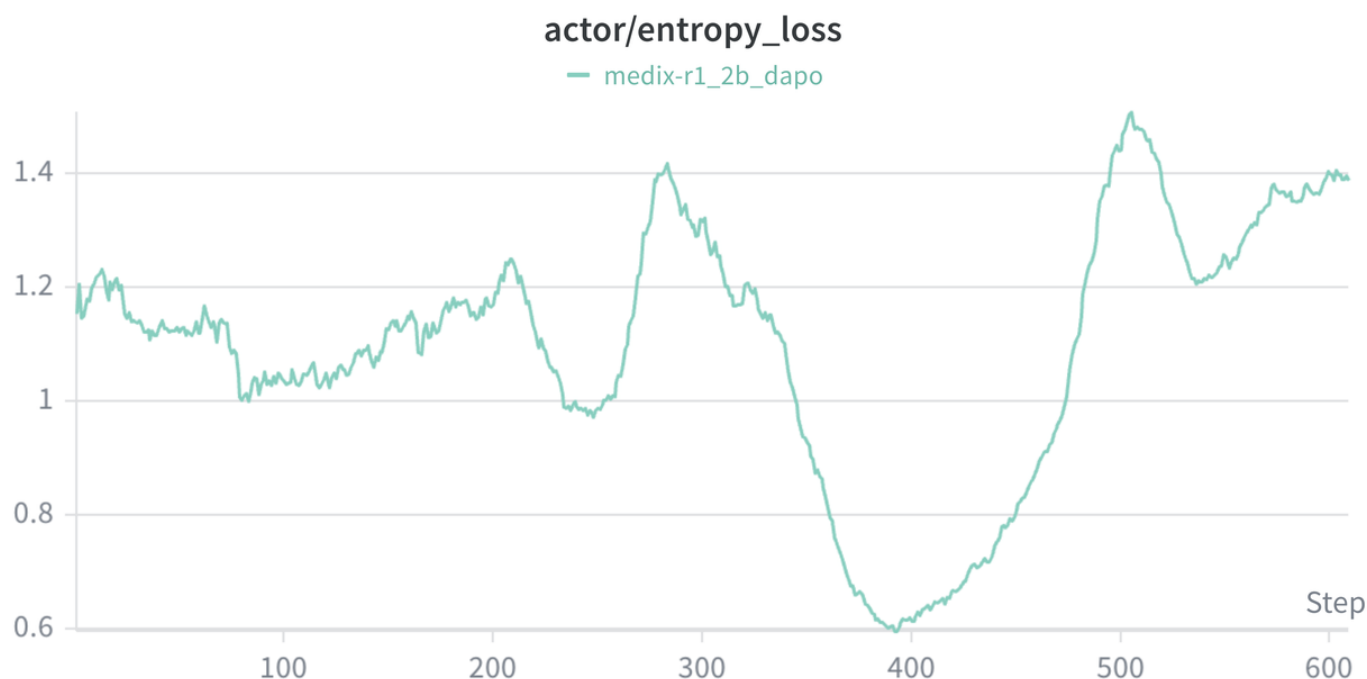


Policy Gradient Loss 是 DAPO 算法的主损失函数，驱动策略优化的方向。

- **初始阶段 (step 0~50):** 损失从 ~0.10 快速下降，策略在剧烈调整
- **中期 (step 50~300):** 损失在 0~0.03 之间波动，偶有小幅跳升
- **后期 (step 300~600):** 损失趋近于 0 附近，振幅进一步减小

解读： pg_loss 从高位收敛到 0 附近是正常且健康的表现 —— 意味着策略更新幅度在缩小，模型逐渐稳定。损失没有出现发散（大幅负值或正值），说明训练过程稳定。

2. 策略熵 (actor/entropy_loss)

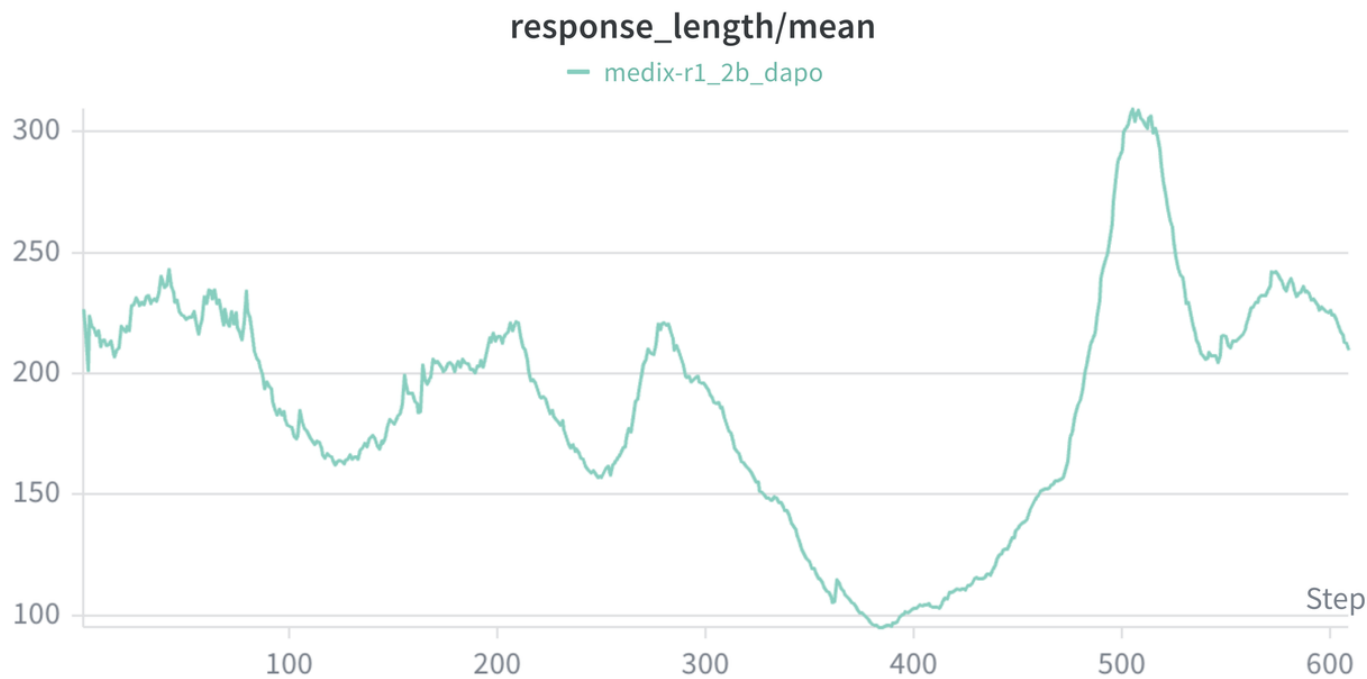


熵反映模型输出的**多样性**，是判断策略是否坍缩的关键信号。

- **初始阶段 (step 0~200)**: 熵从 ~1.15 缓慢下降至 ~1.0，属于正常的策略聚焦过程
- **下降阶段 (step 200~400)**: 熵出现明显下降，从 ~1.4 急剧跌至 ~0.6，同时伴随响应长度的缩短
- **恢复阶段 (step 400~600)**: 熵从 ~0.6 反弹回 ~1.4，恢复到较高水平

解读: step 200~400 期间出现了一次**策略坍缩的趋势** —— 模型开始生成更短、更单一的回答（熵下降 + 响应长度缩短）。但 DAPO 的 online filtering 机制和 clip ratio 约束帮助模型在 step 400 后恢复了多样性。这次"V 形"波动值得关注：如果训练继续，需要监控是否会再次出现类似坍缩。

3. 响应长度 (response_length/mean)



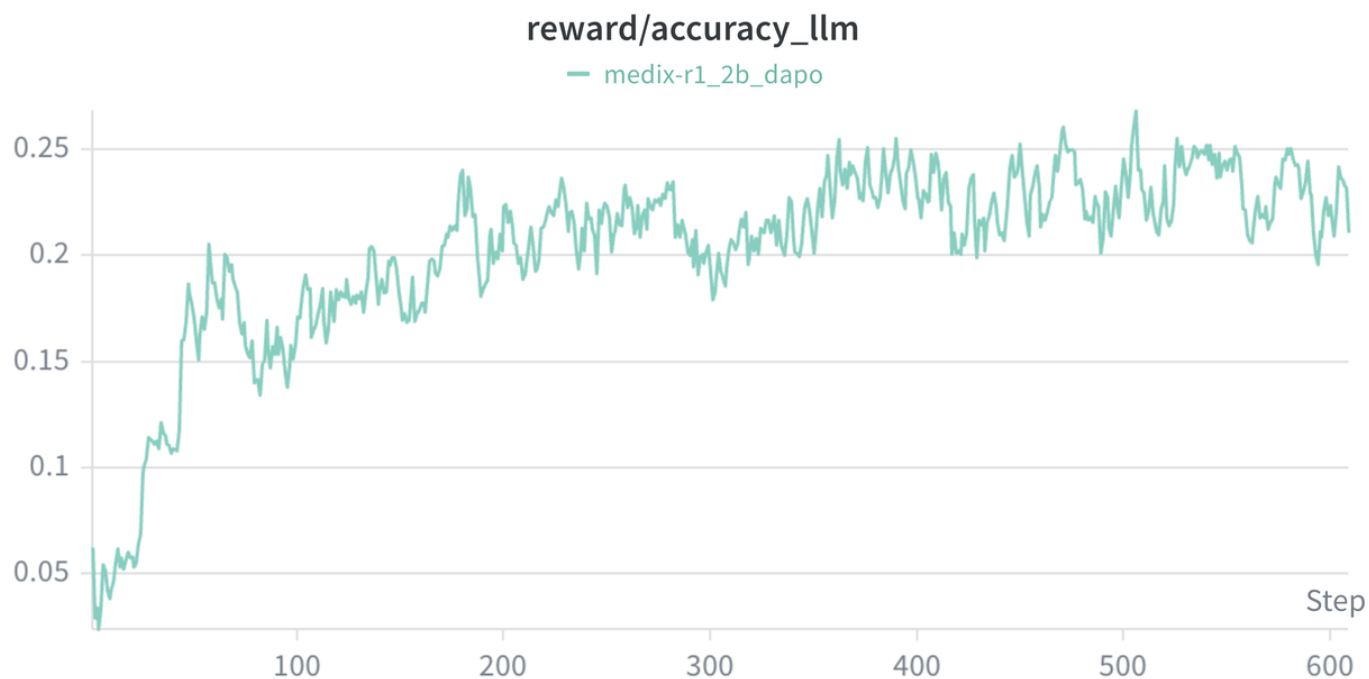
平均响应长度反映模型的输出行为模式变化。

- **初始阶段 (step 0~100)**: 长度从 ~220 逐渐下降至 ~165
- **中期下降 (step 200~400)**: 长度持续下降至最低 ~95 token
- **后期恢复 (step 400~550)**: 长度急剧反弹至 ~305，随后回落稳定在 ~220

解读: 响应长度的变化趋势与熵曲线高度一致。step 200~400 期间模型倾向于生成更简短的回答（可能是找到了一种"偷懒"策略来获取格式分数），随后通过 DAPO 的自我纠正机制恢复了正常长度。最终稳定在 ~220 token 是合理的。

4. 奖励分项分析

4.1 LLM 准确性奖励 (reward/accuracy_llm)

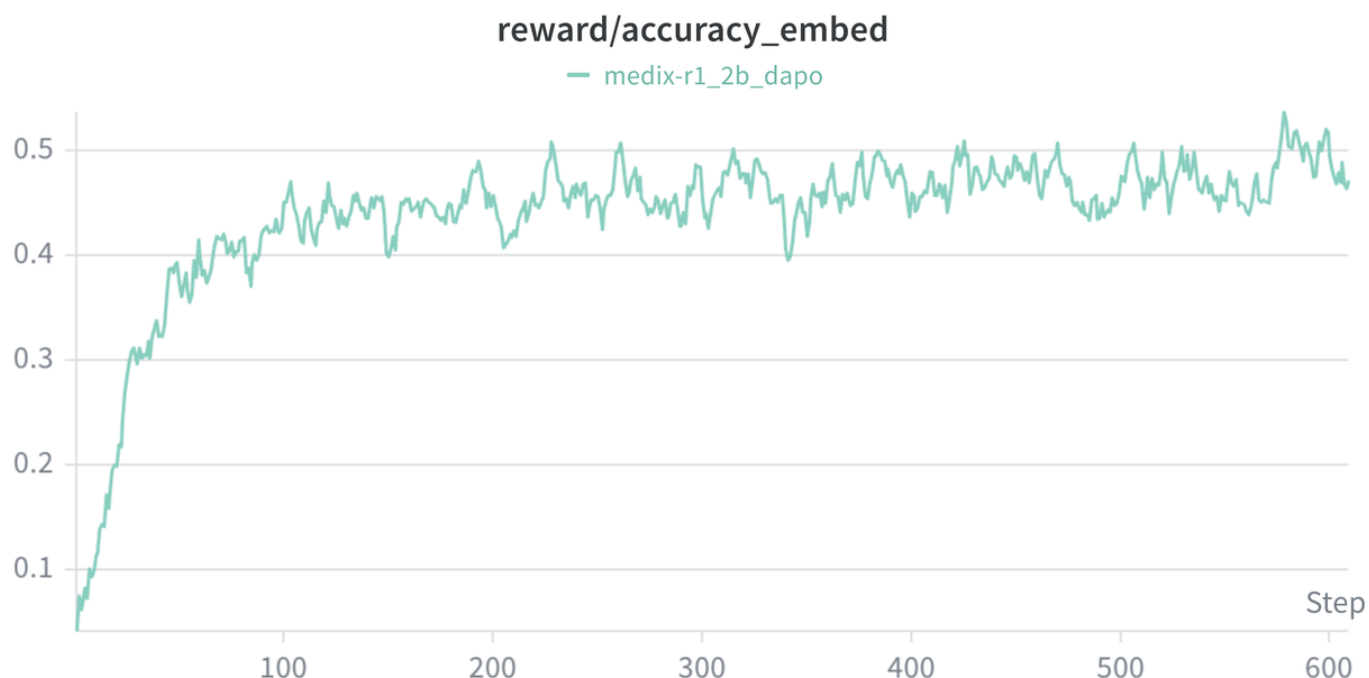


由大模型评判回答准确性的分数。

- 从初始的 ~0.03 上升至 ~0.22 (step 150)，之后稳定在 0.20~0.25 区间
- 整体呈缓慢上升趋势，后半段略高于前半段

解读：LLM 评判的准确性有明显提升，但绝对值仍较低 (~0.22)，说明 2B 模型在医学问答准确性上还有很大提升空间。

4.2 嵌入相似度奖励 (reward/accuracy_embed)



模型回答与标准答案在 MedEmbed 嵌入空间中的语义相似度。

- 从 ~0.03 快速上升至 ~0.45 (step 150)，之后稳定在 0.43~0.50 区间
- 上升速度和最终值都高于 LLM 准确性奖励

解读：模型更快地学会了生成语义上接近标准答案的回答。这说明模型已具备基本的医学语义理解能力。

4.3 格式奖励 (reward/format)

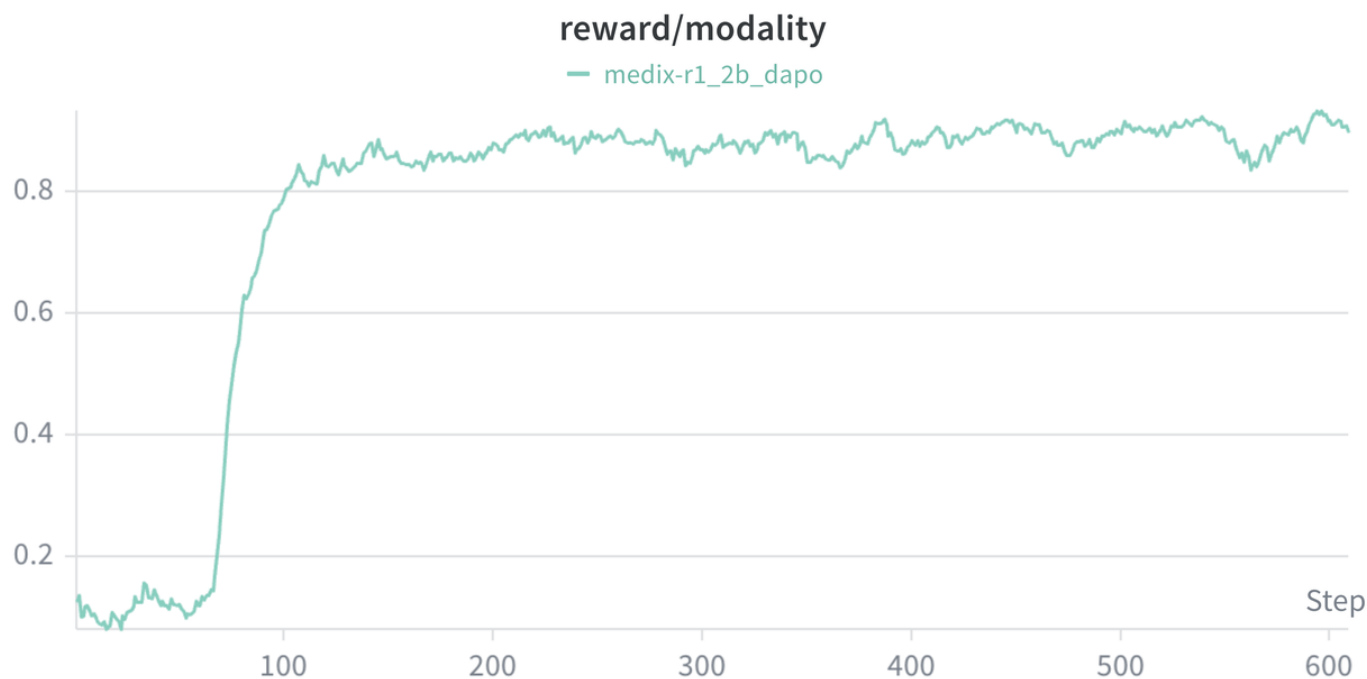


检查回答是否符合 `<think>...</think>` 格式要求。

- 从 ~0.90 迅速上升至 ~1.0 (step 100)，之后几乎保持满分
- 仅在 step ~300 附近有一次微小波动

解读：格式遵循是最容易学习的任务，模型在很早期就掌握了正确的输出格式。这个奖励信号已经饱和，不再对策略优化提供有效梯度。

4.4 模态匹配奖励 (reward/modality)

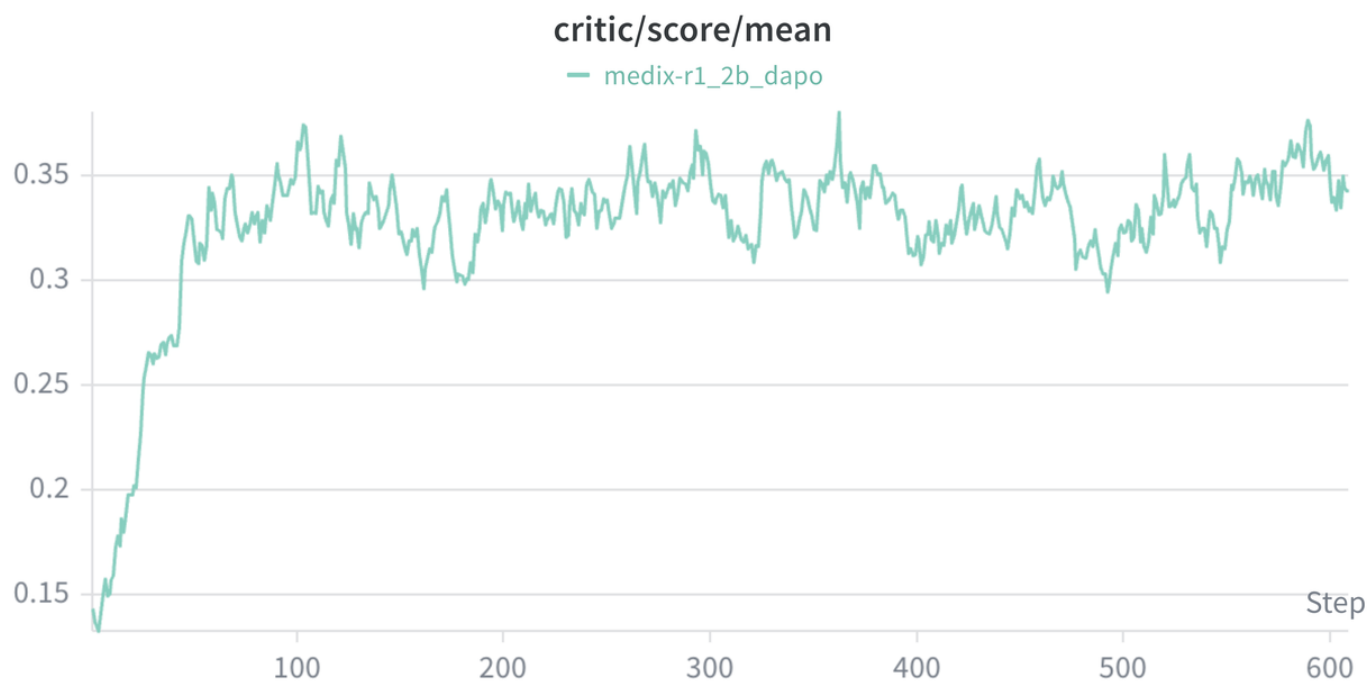


检查模型是否正确识别了输入图像的医学模态（如 X-ray、CT、MRI 等）。

- **突变阶段 (step 70~130):** 从 ~0.10 跃升至 ~0.85，出现了一个明显的阶跃
- **稳定阶段 (step 130~600):** 维持在 0.85~0.92 区间

解读：模态识别能力出现了"顿悟"式的学习 —— 模型在 step 70~130 突然学会了正确描述图像模态。这种阶跃式学习在 RL 训练中是典型现象，说明模型发现了某种有效的模态识别策略。

4.5 综合奖励 (critic/score/mean)



`critic/score/mean` 是每个 rollout batch 中所有样本 `reward/overall` 的**算术平均值**。具体来说，reward function 为每条样本计算一个 `overall` 分数（即上述 4 个分项的加权和），写入 `token_level_scores`；训练框架随后对整个 batch 的 `token_level_scores` 求和再取均值，得到 `critic/score/mean`。因此该指标反映的是模型在**一批数据上的平均综合表现**，而非单条样本的得分。

各分项在 `overall` 中的权重如下（`format_weight=0.1`，`content_weight=0.9`）：

分项	权重
accuracy_llm	$0.9 \times 0.575 = \mathbf{51.75\%}$
accuracy_embed	$0.9 \times 0.375 = \mathbf{33.75\%}$
format	10%
modality	$0.9 \times 0.05 = \mathbf{4.5\%}$

- **起始阶段 (step 0~100)**：奖励从 ~0.13 快速上升至 ~0.35，模型在训练初期迅速学到基本的回答策略
- **平稳阶段 (step 100~600)**：奖励在 0.30~0.38 区间波动，均值稳定在 ~0.34 附近

解读：模型在前 100 步完成了大部分学习，之后进入平台期。奖励没有继续显著上升，说明当前奖励函数的信号可能已被 2B 模型充分利用，或需要更多训练步数/更大模型才能进一步提升。

代码解读（easyR1：简化版 verl）

参考代码：<https://github.com/hiyouga/easyr1>

整个训练流程为：

1. 启动入口：解析命令行配置，初始化 Ray 集群，启动 Runner
2. `Runner.run()`：初始化 tokenizer/processor，定义 Worker 角色映射和 GPU 资源池，创建 reward 函数、数据加载器、Trainer
3. `Trainer.init_workers()`：创建 Ray 资源池，通过 FSDP Worker 初始化 Actor/Rollout/Ref/Critic 模型
4. `Trainer.fit()` 训练循环，每一步：
 - a. Rollout 生成：用 vLLM 对 prompt 采样多个 response
 - b. 计算 `old_log_probs`（当前策略）、`ref_log_probs`（参考策略）、`values`（Critic，如果有的话）
 - c. 计算 reward：通过自定义 reward 函数（支持 LLM-as-judge）给每个 response 打分
 - d. 计算 advantage：支持 GRPO、GAE、RLOO、REINFORCE++、ReMax 等多种估计器
 - e. 更新 Critic（如果使用 GAE）
 - f. 更新 Actor：用 PPO clipped objective 更新策略

g. 定期验证和保存 checkpoint

训练入口

- `training/verl/trainer/main.py`:

`main()` 通过 OmegaConf 解析配置后，初始化 Ray 集群并创建 Runner。Runner.run() 是实际的启动逻辑，做了四件事：

1. 实例化 tokenizer 和 processor（对于 VLM 模型，processor 负责处理图像/视频输入）
2. 定义角色到 Worker 类的映射和 GPU 资源池
3. 创建 reward 函数
4. 创建数据加载器并启动 Trainer

代码块

```
1  @ray.remote(num_cpus=1)
2  class Runner:
3      """A runner for RL training."""
4
5      def run(self, config: PPOConfig):
6          # print config
7          print(json.dumps(config.to_dict(), indent=2))
8
9          # instantiate tokenizer
10         tokenizer = get_tokenizer(
11             config.worker.actor.model.model_path,
12             override_chat_template=config.data.override_chat_template,
13             trust_remote_code=config.worker.actor.model.trust_remote_code,
14             use_fast=True,
15         )
16         processor = get_processor(
17             config.worker.actor.model.model_path,
18             override_chat_template=config.data.override_chat_template,
19             trust_remote_code=config.worker.actor.model.trust_remote_code,
20             use_fast=True,
21         )
22
23         # define worker classes
24         # ActorRolloutRef表示 Actor、Rollout、Ref 三个角色共享同一个 Worker 进程
25         # (即 hybrid engine 模式)。同一个 FSDP 模型在训练时做 Actor，推理时做 Rollout (通过
26         # vLLM)，计算参考 log_probs 时做 Ref。所有角色放在同一个 GPU 资源池中
27         ray_worker_group_cls = RayWorkerGroup
28         role_worker_mapping = {
29             Role.ActorRolloutRef: ray.remote(FSDPWorker),
30             Role.Critic: ray.remote(FSDPWorker),
31         }
```

```

30     global_pool_id = "global_pool"
31     resource_pool_spec = {
32         global_pool_id: [config.trainer.n_gpus_per_node] *
config.trainer.nnodes,
33     }
34     mapping = {
35         Role.ActorRolloutRef: global_pool_id,
36         Role.Critic: global_pool_id,
37     }
38     resource_pool_manager =
ResourcePoolManager(resource_pool_spec=resource_pool_spec, mapping=mapping)
39     # reward 函数被包装成 Ray remote actor, 训练和验证各一个实例。这样 reward 计
算可以和模型计算异步并行。
40     RemoteRewardManager =
ray.remote(AutoRewardManager).options(num_cpus=config.worker.reward.num_cpus)
41     reward_fn = RemoteRewardManager.remote(config.worker.reward, tokenizer)
42     val_reward_fn = RemoteRewardManager.remote(config.worker.reward,
tokenizer)
43
44     train_dataloader, val_dataloader = create_dataloader(config.data,
tokenizer, processor)
45
46     trainer = RayPPOTrainer(
47         config=config,
48         tokenizer=tokenizer,
49         processor=processor,
50         train_dataloader=train_dataloader,
51         val_dataloader=val_dataloader,
52         role_worker_mapping=role_worker_mapping,
53         resource_pool_manager=resource_pool_manager,
54         ray_worker_group_cls=ray_worker_group_cls,
55         reward_fn=reward_fn,
56         val_reward_fn=val_reward_fn,
57     )
58     trainer.init_workers()
59     trainer.fit()

```

数据准备

- `verl/trainer/data_loader.py` & `verl/utils/dataset.py` :

`create_dataloader()` 创建 RLHFDataset 实例, 然后包装成 StatefulDataLoader (支持 checkpoint 恢复时恢复迭代器状态) :

代码块

```

1  def create_dataloader(config: DataConfig, tokenizer, processor):

```

```

2     train_dataset = RLHFDataset(
3         data_path=config.train_files,
4         tokenizer=tokenizer,
5         processor=processor,
6         prompt_key=config.prompt_key,
7         answer_key=config.answer_key,
8         image_key=config.image_key,
9         max_prompt_length=config.max_prompt_length,
10        ...
11    )
12    if config.shuffle:
13        # 使用带 seed 的 RandomSampler 而不是 DataLoader 的 shuffle=True, 是为了
        checkpoint 恢复时能精确还原数据顺序
14        sampler = RandomSampler(data_source=train_dataset,
generator=train_data_loader_generator)
15    else:
16        sampler = SequentialSampler(data_source=train_dataset)
17
18    train_data_loader = StatefulDataLoader(
19        dataset=train_dataset,
20        batch_size=train_batch_size,
21        sampler=sampler,
22        num_workers=8,
23        collate_fn=collate_fn,
24        drop_last=True,
25    )
26

```

重点关注 `getitem()` 函数：首先调用 `_build_messages` 构建多模态 messages，将 prompt 文本和 `<image>` 占位符组合成训练格式。

代码块

```

1  def _build_messages(self, example):
2      prompt_str = example[self.prompt_key]
3      if self.format_prompt:
4          format_prompt = Template(self.format_prompt.strip())
5          prompt_str = format_prompt.render(content=prompt_str)
6
7      if self.image_key in example:
8          content_list = []
9          for i, content in enumerate(prompt_str.split("<image>")):
10             if i != 0:
11                 content_list.append({"type": "image"})
12             if content:
13                 content_list.append({"type": "text", "text": content})

```

```
14         return [{"role": "user", "content": content_list}]
```

接下来用 processor 处理图像，进行图片resize并 tokenize：

代码块

```
1  if self.image_key in example:
2      prompt = self.processor.apply_chat_template(messages,
3          add_generation_prompt=True, tokenize=False)
4      images = example.pop(self.image_key)
5      if self.image_dir is not None and len(images) != 0 and
6          isinstance(images[0], str): # image paths
7          images = [os.path.join(self.image_dir, image) for image in images]
8
9      processed_images = [] if len(images) != 0 else None # text-only data
10     for image in images:
11         processed_images.append(process_image(image, self.min_pixels,
12             self.max_pixels))
13
14     model_inputs = self.processor(processed_images, [prompt],
15         add_special_tokens=False, return_tensors="pt")
16     input_ids = model_inputs.pop("input_ids")[0]
17     attention_mask = model_inputs.pop("attention_mask")[0]
18     example["multi_modal_data"] = {"images": images}
```

应用MRoPE，原理详见 [目 多模态体系知识搭建](#)，交错MRoPE 和 DeepStack代码详见 [目 多模态体系知识搭建](#)

代码块

```
1  if self.processor is not None and "Qwen2VLImageProcessor" in
2      self.processor.image_processor.__class__.__name__:
3      # qwen-vl mrope
4      if "Qwen3VLProcessor" in self.processor.__class__.__name__:
5          from ..models.transformers.qwen3_vl import get_rope_index
6      else:
7          from ..models.transformers.qwen2_vl import get_rope_index
8
9      vision_position_ids = get_rope_index(
10         self.processor,
11         input_ids=input_ids,
12         image_grid_thw=model_inputs.get("image_grid_thw", None),
13         video_grid_thw=model_inputs.get("video_grid_thw", None),
14         second_per_grid_ts=model_inputs.get("second_per_grid_ts", None),
15         attention_mask=attention_mask,
16     ) # (3, seq_length)
```

```

16     text_position_ids = torch.arange(len(input_ids)).unsqueeze(0) # (1,
    seq_length)
17     position_ids = torch.cat((text_position_ids, vision_position_ids), dim=0)
    # (4, seq_length)

```

左填充和截断：

代码块

```

1  input_ids, attention_mask, position_ids = VF.postprocess_data(
2      input_ids=input_ids, attention_mask=attention_mask,
    position_ids=position_ids,
3      max_length=self.max_prompt_length,
    pad_token_id=self.tokenizer.pad_token_id,
4      left_pad=True, truncation="right",
5  )

```

使用左填充是为了和自回归生成对齐——生成时 prompt 在左侧，response 在右侧依次生成，避免 prompt 和 response 之间有 PAD。右截断表示当序列超过 max_prompt_length 时从右侧截掉，保留 prompt 的开头部分。举个例子，假设 max_length=6，原始 token 为 [A, B, C, D, E, F, G, H]：

- 右截断：先截掉右侧超出部分 → [A, B, C, D, E, F]
- 左填充：如果序列不足 max_length，在左侧补 pad → [PAD, PAD, A, B, C, D]

训练器

- `verl/trainer/ray_trainer.py`：trainer 初始化。
 - 如果 `disable_kl=True`（比如 DAPO 算法），就不需要参考策略和 KL 惩罚
 - 只有使用 GAE 作为 advantage estimator 时才需要 Critic 网络。GRPO、RLOO 等算法不需要 Critic

代码块

```

1  class RayPPOTrainer:
2      def __init__(self, config, tokenizer, processor, train_dataloader,
    val_dataloader, ...):
3      # 根据算法配置决定是否使用 KL 控制和 Critic if config.algorithm.disable_kl:
4          self.use_reference_policy = False
5          self.kl_ctrl = FixedKLController(init_kl_coef=0.0)
6      else:
7          self.use_reference_policy = True
8          self.kl_ctrl = get_kl_controller(config.algorithm)
9

```

```

10         if config.algorithm.adv_estimator == AdvantageEstimator.GAE:
11             self.use_critic = True
12             self.use_critic = False

```

init_workers 函数

负责创建分布式 Worker。resource_pool_spec 描述了集群的 GPU 拓扑：key 是资源池名称，value 是一个列表，每个元素表示一个节点上分配多少 GPU。mapping 把训练角色（ActorRolloutRef、Critic）映射到具体的资源池。本项目所有的角色模型共享一组 GPU。

代码块

```

1  def create_resource_pool(self):
2      for resource_pool_name, process_on_nodes in
        self.resource_pool_spec.items():
3          resource_pool = RayResourcePool(
4              process_on_nodes=process_on_nodes, use_gpu=True,
5              max_colocate_count=1, name_prefix=resource_pool_name
6          )
7          self.resource_pool_dict[resource_pool_name] = resource_pool

```

RayResourcePool 底层通过 Ray Placement Group 来实现 GPU 资源的预留和绑定。可以理解为向 Ray 集群"预订座位"——告诉 Ray：我需要在第一个节点上预留 8 个 GPU，在第二个节点上预留 8 个 GPU... Ray 会保证这些资源在训练过程中不被其他任务抢占。

代码：verl/single_controller/ray/base.py

代码块

```

1  def get_placement_groups(self, strategy: str = "STRICT_PACK", name:
    Optional[str] = None) -> list[PlacementGroup]:
2      if self.pgs is not None:
3          return self.pgs
4
5      pg_name_prefix = (
6          name if name else f"{self.name_prefix}verl_group_{'_'}.join([str(count)
7          for count in self._store])):"
8      )
9      # print(f"pg_name_prefix = {pg_name_prefix}")
10     pg_scheme = [
11         [
12             # {"CPU": 1, "GPU": 1} 代表一个"资源槽位", 表示要预留 1 个 CPU 核心和 1
            # 个 GPU
13             # max_colocate_count=1 表示每个槽位只运行一个 Worker 进程, 这样每个 GPU
            # 对应一个独立的 FSDP 进程, 符合分布式训练的标准配置。

```

```

13         {"CPU": self.max_colocate_count, "GPU": 1} if self.use_gpu else
{"CPU": self.max_colocate_count}
14         for _ in range(process_count)
15     ]
16     for process_count in self._store
17 ]
18
19     lifetime = "detached" if self.detached else None
20
21     pgs = [
22         # strategy="STRICT_PACK" 要求同一个 Placement Group (比如节点 1 的 8 个槽
        位) 必须在同一台物理机器上, 不能跨节点
23         placement_group(bundles=bundles, strategy=strategy,
name=pg_name_prefix + str(idx), lifetime=lifetime)
24         for idx, bundles in enumerate(pg_scheme)
25     ]
26
27     ray.get([pg.ready() for pg in pgs])
28
29     self.pgs = pgs
30     return pgs

```

有了资源池之后，`init_workers()` 将 Worker 类绑定到对应的资源池：

代码块

```

1  def init_workers(self):
2      self.resource_pool_manager.create_resource_pool()
3      # 为每个资源池创建一个空字典, 用于存放该池需要运行的 worker 类
4      self.resource_pool_to_cls = {pool: {} for pool in
self.resource_pool_manager.resource_pool_dict.values()}
5      # 接下来将不同角色的 worker 类注册到对应的资源池字典中
6      # 这样设计支持: 1) 多个角色共享同一资源池 (co-location) 2) 不同角色使用独立资源池
7      if self.hybrid_engine:
8          resource_pool =
self.resource_pool_manager.get_resource_pool(Role.ActorRolloutRef)
9          actor_rollout_ref_cls = RayClassWithInitArgs(
10              cls=self.role_worker_mapping[Role.ActorRolloutRef],
config=self.config.worker, role="actor_rollout_ref"
11          )
12          self.resource_pool_to_cls[resource_pool]["actor_rollout_ref"] =
actor_rollout_ref_cls
13          # 后续对 Critic 和 Reward Model 也做同样的注册操作

```


最后通过 `create_collocated_worker_cls()` 将同一资源池内的多个 Worker 类合并到单个 Ray Actor 中（让它们运行在同一进程内共享 GPU），然后创建 WorkerGroup 并部署到对应的资源池上。这样既支持多角色共享 GPU（节省资源），也支持不同角色部署到独立资源池（提高并行度）。



资源分配这里有点抽象，举一个例子你就懂了，想象你在管理一个工厂：

- **节点 (Node)**：一台物理机器，比如你有 2 台服务器
- **GPU**：每台机器上的工人，比如每台服务器有 8 张 GPU 卡
- **进程 (Process)**：一个工作组，一个工作组可以使用一个 GPU
- **集群 (Cluster)**：所有机器联合起来就是一个集群

以下以（**2 节点 × 8 GPU** 的集群 = 2 台服务器，每台服务器有 8 个 GPU，总共 16 个 GPU）举例。

1. 首先是划分资源池（哪些 GPU 给谁用）：

```
resource_pool_spec = {  
    "global_pool": [8, 8] # 第一台机器 8 个 GPU，第二台机器 8 个 GPU  
}
```

这相当于说：我有一个资源池叫 "global_pool"，它包含 2 台机器，每台 8 个 GPU。

然后通过 `ResourcePoolManager` 向 Ray 集群"预订座位"：

2. 决定角色分工（谁负责干啥）

- **Actor**：负责训练策略模型（学习怎么回答问题）
- **Rollout**：负责生成答案（用 vLLM 快速推理）
- **Ref**：负责当"参照物"（记住初始模型，防止训练跑偏）
- **Critic**：负责评估答案好坏（DAPO、GSPO算法不需要）
- **RewardModel**：负责打分（判断答案质量）

3. 把角色分配到资源池（谁去哪台机器工作）

```
mapping = {  
    Role.ActorRolloutRef: "global_pool",  
    Role.Critic: "global_pool",  
}
```

这里是把 `ActorRolloutRef` 和 `Critic` 都放在一个资源池工作，共享16个GPU。之后在 `resource_pool_to_cls` 中注册所有的模型。

4. `create_collocated_worker_cls`

把 `ActorRolloutRef` 和 `Critic` 打包成一个 Ray Actor，共享同一个进程，最后 `WorkerGroup` 会在每个 GPU 上启动这个合并后的进程。

fit函数

代码: `verl/trainer/ray_trainer.py`

代码块

```
1  with timer("gen", timing_raw):
2      self.actor_rollout_ref_wg.prepare_rollout_engine()
3      batch = self._make_batch_data(metrics=metrics)
4      self.actor_rollout_ref_wg.release_rollout_engine()
```

`prepare_rollout_engine()` 将 FSDP 模型的权重同步到 vLLM 引擎。

`_make_batch_data()` 从数据集取一批 prompt，用 vLLM 生成 response。生成完成后

`release_rollout_engine()` 释放 vLLM 占用的显存，为后续训练腾出空间。

代码块

```
1  # make a batch of data
2  with timer("gen", timing_raw):
3      self.actor_rollout_ref_wg.prepare_rollout_engine()
4      batch = self._make_batch_data(metrics=metrics)
5      self.actor_rollout_ref_wg.release_rollout_engine()
```

`_make_batch_data()` 中实现了 DAPO 中的 **Online Filtering** 思想，也就是过滤掉模型全队或者全错的样本，提高训练效率，避免在无意义的样本上浪费计算资源。

代码块

```
1  def _make_batch_data(self, metrics):
2      batch = None
3      num_try_make_batch = 0
4
5      while True:
6          num_try_make_batch += 1
7
8          # 1. 从数据加载器取一个 mini-batch 的 prompt (例如 32 个 prompt)
9          batch_dict = next(self.data_iterator)
10         new_batch = DataProto.from_single_dict(batch_dict, meta_info=meta_info)
11
12         # 2. 为每个 prompt 生成一个唯一的 uid (用于 online filtering 时追踪同一
            prompt 的多个 response)
```

```

13         new_batch.non_tensor_batch["uid"] = np.array([str(uuid.uuid4()) for _
14         in range(len(new_batch.batch))])
15
16         # 3. 提取生成所需的数据
17         gen_batch = new_batch.pop(
18             batch_keys=["input_ids", "attention_mask", "position_ids"],
19             non_tensor_batch_keys=["raw_prompt_ids", "multi_modal_data"],
20         )
21
22         # 4. 调用 vLLM 生成 response (如果 rollout.n=4, 每个 prompt 会生成 4 个不同
23         的 response)
24         gen_batch_output =
25         self.actor_rollout_ref_wg.generate_sequences(gen_batch)
26
27         # 5. 把 prompt 侧的数据 (ground_truth 等) 重复 n 次, 对齐生成的 n 个
28         response
29         # 例如: 原本 32 个 prompt, rollout.n=4, 重复后变成 128 条数据 (32
30         prompt × 4 response)
31         new_batch =
32         new_batch.repeat(repeat_times=self.config.worker.rollout.n, interleave=True)
33         new_batch = new_batch.union(gen_batch_output)
34
35         # 6. Online Filtering (DAPO 的关键特性)
36         if self.config.algorithm.online_filtering:
37             # 6.1 立即调用 reward 函数对所有 response 打分
38             reward_tensor, reward_metrics =
39             ray.get(self.reward_fn.compute_reward.remote(new_batch))
40             new_batch.batch["token_level_scores"] = reward_tensor
41
42             # 6.2 根据指定的 filter_key (例如 "overall" 或 "accuracy") 提取分数
43             filter_scores = reward_metrics[self.config.algorithm.filter_key]
44             # 长度 128 (32×4)
45             uids = new_batch.non_tensor_batch["uid"] # 长度 128, 但每 4 个有相同
46             的 uid
47
48             # 6.3 按 uid 分组, 计算每个 prompt 的平均分数
49             # 例如: prompt_A 的 4 个 response 得分 [0.2, 0.3, 0.25, 0.28], 平
50             均分 0.2575
51             uid2scores = defaultdict(list)
52             for uid, score in zip(uids, filter_scores):
53                 uid2scores[uid].append(score)
54             uid2mean = {uid: np.mean(scores) for uid, scores in
55             uid2scores.items()}
56
57             # 6.4 过滤掉"太简单"或"太难"的样本
58             # - filter_low: 例如 0.8, 过滤掉平均分 > 0.8 的 (太简单, 模型已经学
59             会了)

```

```

48         # - filter_high: 例如 0.2, 过滤掉平均分 < 0.2 的 (太难, 噪音样本)
49         # 保留 0.2 < avg_score < 0.8 的样本 (难度适中, 对训练有价值)
50         kept_uids = [
51             uid for uid, avg_score in uid2mean.items()
52             if avg_score > self.config.algorithm.filter_low
53             and avg_score < self.config.algorithm.filter_high
54         ]
55
56         # 6.5 根据 kept_uids 筛选出保留的样本索引
57         # 如果 32 个 prompt 中只有 20 个被保留, 则 kept_sample_idx 长度为
80 (20×4)
58         kept_sample_idx = [idx for idx, uid in enumerate(uids) if uid in
kept_uids]
59         new_batch = new_batch[kept_sample_idx]
60
61         # 7. 累积到总 batch 中
62         batch = DataProto.concat([batch, new_batch]) if batch is not None else
new_batch
63         current_batch_size = len(batch) // self.config.worker.rollout.n
64
65         # 8. 检查是否凑够了目标数量 (例如目标 rollout_batch_size=256)
66         # - 如果 online filtering 过滤掉很多样本, current_batch_size 可能不足
67         # - 此时继续循环, 再取下一批 prompt 生成, 直到凑够 256 个 prompt
68         rollout_batch_size = self.config.data.rollout_batch_size
69         if current_batch_size >= rollout_batch_size:
70             # 凑够了, 返回前 rollout_batch_size * n 条数据 (例如 256×4=1024)
71             return batch[:rollout_batch_size * self.config.worker.rollout.n]
72         else:
73             # 不够, 继续生成 (但设置最大尝试次数 max_try_make_batch 防止死循环)
74             if num_try_make_batch >= self.config.trainer.max_try_make_batch:
75                 raise RuntimeError(f"尝试了 {num_try_make_batch} 次仍未凑够数据,
请检查过滤阈值")

```

在分布式训练中, 如果不同 GPU 上分到的序列长度差异很大, 短序列的 GPU 会等长序列的 GPU, 造成浪费。 `_balance_batch()` 在 driver 端重新排列数据, 让每个 GPU 分到的总 token 数尽量接近。

代码块

```

1  def _balance_batch(self, batch: DataProto, metrics: dict[str, Any],
logging_prefix: str = "global_seqlen") -> None:
2      """Reorder the data on single controller such that each dp rank gets
similar total tokens"""
3      attention_mask = batch.batch["attention_mask"]
4      batch_size = attention_mask.shape[0]

```

```

5     global_seqlen_lst = batch.batch["attention_mask"].view(batch_size,
-1).sum(-1).tolist() # (train_batch_size,)
6     world_size = self.actor_rollout_ref_wg.world_size
7     global_partition_lst = get_seqlen_balanced_partitions(
8         global_seqlen_lst, k_partitions=world_size, equal_size=True
9     )
10    # reorder based on index. The data will be automatically equally
    partitioned by dispatch function
11    global_idx = torch.tensor([j for partition in global_partition_lst for j in
    partition])
12    batch.reorder(global_idx)
13    global_balance_stats = log_seqlen_unbalance(
14        seqlen_list=global_seqlen_lst, partitions=global_partition_lst,
    prefix=logging_prefix
15    )
16    metrics.update(global_balance_stats)

```

同时计算 log_probs 和 reward，后面 `ray.get()` 取回结果

代码块

```

1  # 异步计算奖励
2  reward_ref = self.reward_fn.compute_reward.remote(batch)
3  # 计算当前策略的 log_probs
4  old_log_probs = self.actor_rollout_ref_wg.compute_log_probs(batch)
5  batch = batch.union(old_log_probs)
6
7  # 计算参考策略的 log_probs (用于 KL 惩罚)
8  if self.use_reference_policy:
9      ref_log_probs = self.actor_rollout_ref_wg.compute_ref_log_probs(batch)
10     batch = batch.union(ref_log_probs)
11
12  # 计算 Critic 的 value 估计
13  if self.use_critic:
14      values = self.critic_wg.compute_values(batch)
15      batch = batch.union(values)

```

计算 advantage 和更新模型

代码块

```

1  with timer("adv", timing_raw):
2      if "token_level_scores" not in batch.batch:
3          reward_tensor, reward_metrics = ray.get(reward_ref)
4          batch.batch["token_level_scores"] = reward_tensor
5

```

```

6         if not self.config.algorithm.use_kl_loss and self.use_reference_policy:
7             # 先把 KL 惩罚加到 token 级 reward 上, 然后用选定的 advantage estimator 计
              算 advantage。
8             batch, kl_metrics = apply_kl_penalty(batch, self.kl_ctrl,
              self.config.algorithm.kl_penalty)
9         else:
10            batch.batch["token_level_rewards"] = batch.batch["token_level_scores"]
11
12            batch = compute_advantage(batch,
              adv_estimator=self.config.algorithm.adv_estimator, ...)
13
14        if self.use_critic:
15            critic_output = self.critic_wg.update_critic(batch)
16            # critic_warmup 允许先只训练 Critic 若干步, 等 value 估计稳定后再开始更新 Actor。
17        if self.config.trainer.critic_warmup <= self.global_step:
18            actor_output = self.actor_rollout_ref_wg.update_actor(batch)

```

详解损失函数计算代码

这一步是 强化学习 训练的核心准备阶段，需要计算三个关键量：

1. **old_log_probs**（当前策略的 log 概率）
2. **ref_log_probs**（参考策略的 log 概率，用于 KL 惩罚）
3. **values**（Critic 网络的价值估计，仅 GAE 算法需要）

计算 old_log_probs

代码位置： `verl/workers/fsdp_workers.py`

代码块

```

1  @register(dispatch_mode=Dispatch.DP_COMPUTE_PROTO)
2  def compute_log_probs(self, data: DataProto):
3      # 1. 处理多模态输入（图像等）
4      self._process_multi_modal_inputs(data)
5      data = data.to(torch.cuda.current_device())
6
7      # 2. 如果启用了参数 offload, 从 CPU 加载模型到 GPU
8      if self._use_param_offload:
9          load_fsdp_model(self.fsdp_module)
10
11      # 3. 设置温度参数（用于控制采样的随机性）
12      data.meta_info["temperature"] = self.config.rollout.temperature
13
14      # 4. 序列并行预处理
15      with self.ulysses_sharding_manager:
16          data = self.ulysses_sharding_manager.preprocess_data(data)

```

```

17         output = self.actor.compute_log_prob(data=data)
18         output = DataProto.from_dict(
19             tensors={"old_log_probs": output},
20             meta_info={"temperature": self.config.rollout.temperature}
21         )
22         output = self.ulysses_sharding_manager.postprocess_data(output)
23
24         # 5. FSDP reshard (多 GPU 时需要)
25         if self.world_size > 1:
26             self.fsdp_module._handle.reshard(True)
27
28         # 6. 计算完成后 offload 回 CPU
29         if self._use_param_offload:
30             offload_fsdp_model(self.fsdp_module)
31
32         return output.to("cpu")

```

核心计算在 `self.actor.compute_log_prob()`，位于 `verl/workers/actor/dp_actor.py`：

代码块

```

1  def compute_log_prob(self, data: DataProto) -> torch.Tensor:
2      self.actor_module.eval() # 设置为评估模式
3
4      # 1. 提取需要的字段
5      select_keys = ["input_ids", "attention_mask", "position_ids", "responses"]
6      non_tensor_select_keys = ["multi_modal_inputs"]
7      data = data.select(select_keys, non_tensor_select_keys)
8
9      # 2. 动态 batching 或固定 micro-batch
10     if self.config.dynamic_batching:
11         # 按总 token 数切分, 保证每个 micro-batch 的计算量相近
12         max_token_len = self.config.micro_batch_size_per_device_for_experience
13         * data.batch["input_ids"].size(-1)
14         micro_batches, batch_idx_list = prepare_dynamic_batch(data,
15             max_token_len=max_token_len)
16     else:
17         # 按固定的 batch size 切分
18         micro_batches =
19         data.split(self.config.micro_batch_size_per_device_for_experience)
20
21     # 3. 逐个 micro-batch 前向计算
22     log_probs_lst = []
23     for micro_batch in micro_batches:
24         model_inputs = {**micro_batch.batch, **micro_batch.non_tensor_batch}

```

```

22         log_probs = self._forward_micro_batch(model_inputs,
temperature=temperature)
23         log_probs_lst.append(log_probs)
24
25     # 4. 拼接所有 micro-batch 的结果
26     log_probs = torch.concat(log_probs_lst, dim=0)
27
28     # 5. 恢复原始顺序 (如果用了动态 batching)
29     if self.config.dynamic_batching:
30         log_probs = restore_dynamic_batch(log_probs, batch_idx_list)
31
32     return log_probs

```

`_forward_micro_batch()` 的核心逻辑:

代码块

```

1  def _forward_micro_batch(self, micro_batch: dict, temperature: float) ->
torch.Tensor:
2      input_ids = micro_batch["input_ids"] # (bs, prompt_len + response_len)
3      attention_mask = micro_batch["attention_mask"]
4      position_ids = micro_batch["position_ids"]
5      responses = micro_batch["responses"] # (bs, response_len)
6      response_length = responses.size(-1)
7
8      # ===== 模式1: Padding-free (使用 flash_attn 的 unpad/pad) =====
9      if self.config.padding_free:
10         # 1. 去除 padding, 将所有有效 token 拼成一维序列
11         input_ids_rmpad, indices, *_ = unpad_input(input_ids.unsqueeze(-1),
attention_mask)
12         input_ids_rmpad = input_ids_rmpad.transpose(0, 1) # (1, total_nnz)
13
14         # 2. 处理 position_ids
15         if position_ids.dim() == 3: # M-RoPE
16             position_ids_rmpad = index_first_axis(
17                 rearrange(position_ids, "c b s ... -> (b s) c ..."), indices
18             ).transpose(0, 1).unsqueeze(1) # (4, 1, total_nnz)
19         else:
20             position_ids_rmpad = index_first_axis(position_ids, indices)
21
22         # 3. 前向传播 (attention_mask=None, flash_attn 会自动处理)
23         output = self.actor_module(
24             input_ids=input_ids_rmpad,
25             attention_mask=None,
26             position_ids=position_ids_rmpad,
27             **multi_modal_inputs,

```



```

28         use_cache=False,
29     )
30
31     # 4. 提取 response 部分的 logits
32     logits = output.logits[:, -response_length - 1 : -1, :] # (1,
response_len, vocab_size)
33     logits.div_(temperature) # 温度缩放
34
35     # 5. 计算 log_probs
36     responses_rmpad = index_first_axis(responses, indices)
37     log_probs = self.log_probs_from_logits(logits, responses_rmpad)
38
39     # 6. Pad 回原始形状
40     full_log_probs = pad_input(log_probs.unsqueeze(-1), indices,
batch=batch_size, seqlen=response_length)
41     return full_log_probs.squeeze(-1)
42
43     # ===== 模式2: 标准 padding =====
44     else:
45         # 1. 前向传播
46         output = self.actor_module(
47             input_ids=input_ids,
48             attention_mask=attention_mask,
49             position_ids=position_ids,
50             **multi_modal_inputs,
51             use_cache=False,
52         )
53
54         # 2. 提取 response 部分的 logits
55         # 因为自回归模型的 logits[t] 预测的是 token[t+1]
56         # 所以取 [-response_length - 1 : -1]
57         logits = output.logits[:, -response_length - 1 : -1, :] # (bs,
response_len, vocab_size)
58         logits.div_(temperature)
59
60         # 3. 计算 log_probs
61         log_probs = self.log_probs_from_logits(logits, responses)
62         return log_probs # (bs, response_len)

```

关键函数 `log_probs_from_logits` 的实现（在 `verl/utils/torch_functional.py`）：

代码块

```

1 def log_probs_from_logits(logits: torch.Tensor, labels: torch.Tensor) ->
torch.Tensor:

```

```

2      """
3      Args:
4          logits: (bs, seq_len, vocab_size)
5          labels: (bs, seq_len)
6      Returns:
7          log_probs: (bs, seq_len)
8      """
9      log_probs = F.log_softmax(logits, dim=-1) # (bs, seq_len, vocab_size)
10     # gather 操作: 从 vocab 维度选出 labels 对应的 log_prob
11     log_probs_labels = torch.gather(log_probs, dim=-1,
12                                     index=labels.unsqueeze(-1)) # (bs, seq_len, 1)
13     return log_probs_labels.squeeze(-1) # (bs, seq_len)

```

公式对应:

$$\pi_{\theta}(a_t|s_t) = \frac{\exp(\text{logits}[a_t]/T)}{\sum_{a'} \exp(\text{logits}[a']/T)}$$

$$\log \pi_{\theta}(a_t|s_t) = \log(\text{softmax}(\text{logits}/T)[a_t])$$

计算 ref_log_probs

代码位置: `verl/workers/fsdp_workers.py`

代码块

```

1  @register(dispatch_mode=Dispatch.DP_COMPUTE_PROTO)
2  def compute_ref_log_probs(self, data: DataProto):
3      self._process_multi_modal_inputs(data)
4      data = data.to(torch.cuda.current_device())
5
6      # 从 CPU 加载冻结的参考模型
7      if self._use_ref_param_offload:
8          load_fsdp_model(self.ref_fsdp_module)
9
10     data.meta_info["temperature"] = self.config.rollout.temperature
11     with self.ulysses_sharding_manager:
12         data = self.ulysses_sharding_manager.preprocess_data(data)
13         # 调用参考策略的 compute_log_prob (和 actor 一样的逻辑)
14         output = self.ref_policy.compute_log_prob(data=data)
15         output = DataProto.from_dict(tensors={"ref_log_probs": output})
16         output = self.ulysses_sharding_manager.postprocess_data(output)
17
18     # FSDP reshard
19     if self.world_size > 1:
20         self.ref_fsdp_module._handle.reshard(True)

```

```

21
22     # Offload 回 CPU
23     if self._use_ref_param_offload:
24         offload_fsdp_model(self.ref_fsdp_module)
25
26     return output.to("cpu")

```

ref_policy 和 actor 的区别:

- `ref_fsdp_module` 是训练开始时的初始模型权重，**全程冻结不更新**
- 用于计算 KL 散度: $D_{KL}(\pi_{\theta}||\pi_{ref})$ ，防止策略更新偏离初始模型太远

公式对应:

$$\log \pi_{ref}(a_t|s_t) = \log(\text{softmax}(\text{logits}_{ref}/T)[a_t])$$

计算 values (仅 GAE 算法需要)

代码位置: `verl/workers/fsdp_workers.py`

代码块

```

1  @register(dispatch_mode=Dispatch.DP_COMPUTE_PROTO)
2  def compute_values(self, data: DataProto):
3      self._process_multi_modal_inputs(data)
4      data = data.to(torch.cuda.current_device())
5
6      if self._use_param_offload:
7          load_fsdp_model(self.fsdp_module)
8
9      with self.ulysses_sharding_manager:
10         data = self.ulysses_sharding_manager.preprocess_data(data=data)
11         # 调用 Critic 网络计算 value
12         values = self.critic.compute_values(data=data)
13         output = DataProto.from_dict(tensors={"values": values})
14         output = self.ulysses_sharding_manager.postprocess_data(data=output)
15
16     if self._use_param_offload:
17         offload_fsdp_model(self.fsdp_module)
18
19     return output.to("cpu")

```

Critic 的前向计算 (在 `verl/workers/critic/dp_critic.py`):

代码块

```

1  def compute_values(self, data: DataProto) -> torch.Tensor:

```

```

2     self.critic_module.eval()
3
4     # 提取字段
5     select_keys = ["input_ids", "attention_mask", "position_ids"]
6     non_tensor_select_keys = ["multi_modal_inputs"]
7     data = data.select(select_keys, non_tensor_select_keys)
8
9     # 切分 micro-batch
10    micro_batches =
data.split(self.config.micro_batch_size_per_device_for_experience)
11
12    values_lst = []
13    for micro_batch in micro_batches:
14        model_inputs = {**micro_batch.batch, **micro_batch.non_tensor_batch}
15        values = self._forward_micro_batch(model_inputs) # (bs, response_len)
16        values_lst.append(values)
17
18    values = torch.concat(values_lst, dim=0)
19    return values
20
21    def _forward_micro_batch(self, micro_batch: dict) -> torch.Tensor:
22        input_ids = micro_batch["input_ids"]
23        attention_mask = micro_batch["attention_mask"]
24        position_ids = micro_batch["position_ids"]
25        response_length = micro_batch.get("responses").size(-1)
26
27        # 前向传播
28        output = self.critic_module(
29            input_ids=input_ids,
30            attention_mask=attention_mask,
31            position_ids=position_ids,
32            **multi_modal_inputs,
33            use_cache=False,
34        )
35
36        # Critic 模型的输出是一个标量 value head
37        # 取 response 部分的 value 估计
38        values = output.logits[:, -response_length:, 0] # (bs, response_len)
39        return values

```

公式对应：

$$V^{\pi}(s_t) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t \right]$$

KL Penalty

在计算 advantage 之前，先把 KL 惩罚加到 reward 上：

代码块

```
1 if not self.config.algorithm.use_kl_loss and self.use_reference_policy:
2     batch, kl_metrics = apply_kl_penalty(batch, self.kl_ctrl,
3     self.config.algorithm.kl_penalty)
4 else:
5     batch.batch["token_level_rewards"] = batch.batch["token_level_scores"]
```

`apply_kl_penalty()` 的实现，代码在 (`verl/trainer/ray_trainer.py`)：

代码块

```
1 def apply_kl_penalty(batch: DataProto, kl_ctrl: KLController, kl_penalty: str):
2     # 1. 提取字段
3     old_log_probs = batch.batch["old_log_probs"] # (bs, response_len)
4     ref_log_probs = batch.batch["ref_log_probs"]
5     token_level_scores = batch.batch["token_level_scores"]
6     response_mask = batch.batch["response_mask"]
7
8     # 2. 计算 KL 散度
9     kld = compute_kl(log_probs=old_log_probs, ref_log_probs=ref_log_probs,
10     kl_penalty=kl_penalty)
11     # kld shape: (bs, response_len)
12
13     # 3. 加入 KL 惩罚
14     token_level_rewards = token_level_scores - kl_ctrl.kl_coef * kld # (bs,
15     response_len)
16
17     # 4. 计算平均 KL (用于监控和自适应调整)
18     mean_kl = VF.masked_mean(kld, response_mask).item()
19
20     # 5. 更新 KL 系数 (如果是自适应 KL)
21     kl_ctrl.update(current_kl=mean_kl, n_steps=1)
22
23     batch.batch["token_level_rewards"] = token_level_rewards
24     return batch, {"kl_divergence": mean_kl, "kl_coef": kl_ctrl.kl_coef}
```

`compute_kl()` 的实现 (代码在 `verl/trainer/core_algos.py`)：

代码块

```
1 def compute_kl(log_probs: torch.Tensor, ref_log_probs: torch.Tensor,
2     kl_penalty: str) -> torch.Tensor:
3     """
```

```

3     Args:
4         log_probs: (bs, response_len)
5         ref_log_probs: (bs, response_len)
6         kl_penalty: "kl" | "abs" | "mse" | "low_var_kl" | "full"
7     Returns:
8         kl_div: (bs, response_len)
9     """
10    log_probs, ref_log_probs = log_probs.float(), ref_log_probs.float()
11
12    if kl_penalty == "kl":
13        return log_probs - ref_log_probs
14
15    if kl_penalty == "abs":
16        return (log_probs - ref_log_probs).abs()
17
18    if kl_penalty == "mse":
19        return 0.5 * (log_probs - ref_log_probs).square()
20
21    if kl_penalty == "low_var_kl":
22        # Schulman 的低方差 KL 近似:  $\exp(kl) - kl - 1$ 
23        kl = (ref_log_probs - log_probs).clamp(-20.0, 20.0)
24        kld = (kl.exp() - kl - 1).contiguous()
25        return torch.clamp(kld, min=-10.0, max=10.0)
26
27    if kl_penalty == "full":
28        # 完整的 KL 散度 (需要所有 action 的概率分布)
29        return F.kl_div(ref_log_probs, log_probs, log_target=True,
30                        reduction="none").sum(-1)
31
32    raise NotImplementedError(f"Unknown KL penalty: {kl_penalty}.")

```

公式对应：

简化 KL (kl 模式)：

$$KL(a_t) \approx \log \pi_{\theta}(a_t|s_t) - \log \pi_{ref}(a_t|s_t)$$

这是一个偏估计，只考虑了实际采样的 action，没有对整个 action 空间积分。

完整 KL (full 模式)：

$$D_{KL}(\pi_{\theta}||\pi_{ref}) = \sum_a \pi_{\theta}(a|s) \log \frac{\pi_{\theta}(a|s)}{\pi_{ref}(a|s)}$$

低方差 KL (low_var_kl 模式)：

基于 Schulman 的近似公式：

$$D_{KL}(p||q) \approx \exp(D_{KL}(q||p)) - D_{KL}(q||p) - 1$$

KL 惩罚后的 reward:

$$r_t^{KL} = r_t - \beta \cdot \text{KL}(a_t)$$

计算 Advantage

代码位置: `verl/trainer/ray_trainer.py` 的 `fit()` 方法

代码块

```
1 batch = compute_advantage(  
2     batch,  
3     adv_estimator=self.config.algorithm.adv_estimator,  
4     gamma=self.config.algorithm.gamma,  
5     lam=self.config.algorithm.lam,  
6 )
```

`compute_advantage()` 的调度:

代码块

```
1 def compute_advantage(batch: DataProto, adv_estimator: AdvantageEstimator,  
   **kwargs) -> DataProto:  
2     """  
3     根据 adv_estimator 选择不同的 advantage 计算方法  
4     """  
5     # 提取字段  
6     token_level_rewards = batch.batch["token_level_rewards"] # (bs,  
   response_len)  
7     response_mask = batch.batch["response_mask"]  
8  
9     # 调用对应的 advantage estimator  
10    advantages, returns = compute_advantage_return(adv_estimator, **adv_inputs)  
11    # 存回 batch  
12    batch.batch["advantages"] = advantages  
13    batch.batch["returns"] = returns  
14    return batch
```

GAE (Generalized Advantage Estimation)

代码 (`verl/trainer/core_algos.py`) :

代码块

```
1 @register_adv_estimator(AdvantageEstimator.GAE)  
2 def compute_gae_advantage_return(  
3     token_level_rewards: torch.Tensor, # (bs, response_len)  
4     values: torch.Tensor, # (bs, response_len)
```

```

5     response_mask: torch.Tensor,          # (bs, response_len)
6     gamma: torch.Tensor,                 # 折扣因子, 默认 1.0
7     lam: torch.Tensor,                   # GAE 参数, 默认 1.0
8     **kwargs,
9 ) -> tuple[torch.Tensor, torch.Tensor]:
10     nextvalues = 0
11     lastgaelam = 0
12     advantages_reversed = []
13     gen_len = token_level_rewards.shape[-1]
14
15     # 从后往前递推
16     for t in reversed(range(gen_len)):
17         # 1. 计算 TD 误差
18         delta = token_level_rewards[:, t] + gamma * nextvalues - values[:, t]
19
20         # 2. 计算 GAE advantage
21         gaelam = delta + gamma * lam * lastgaelam
22
23         # 3. 只在有效 token 上更新 (跳过 padding 和 EOS 后的 token)
24         if response_mask[:, t]:
25             nextvalues = values[:, t]
26             lastgaelam = gaelam
27
28         advantages_reversed.append(lastgaelam)
29
30     # 4. 反转回正向
31     advantages = torch.stack(advantages_reversed[::-1], dim=1) # (bs,
response_len)
32
33     # 5. 计算 returns
34     returns = advantages + values
35
36     # 6. 对 advantages 做 whitening (归一化)
37     advantages = VF.masked_whiten(advantages, response_mask)
38
39     return advantages, returns

```

公式对应:

TD 误差:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE Advantage:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

递推形式：

$$\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}$$

Returns：

$$R_t = \hat{A}_t + V(s_t)$$

Whitening (归一化)：

$$\hat{A}_t^{normalized} = \frac{\hat{A}_t - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + \epsilon}$$

这一步让 advantage 的分布更稳定，梯度更新更平滑。

GRPO

代码 (`verl/trainer/core_algos.py`)：

代码块

```
1  @register_adv_estimator(AdvantageEstimator.GRPO)
2  def compute_grpo_outcome_advantage(
3      token_level_rewards: torch.Tensor, # (bs, response_len)
4      response_mask: torch.Tensor,      # (bs, response_len)
5      index: torch.Tensor,              # (bs,), 标识哪些 response 来自同一个
    prompt
6      eps: float = 1e-6,
7      **kwargs,
8  ) -> tuple[torch.Tensor, torch.Tensor]:
9      # 1. 计算每个 response 的总分数 (outcome reward)
10     scores = token_level_rewards.sum(dim=-1) # (bs,)
11
12     # 2. 按 index 分组，收集同一个 prompt 的所有 response 的分数
13     id2score = defaultdict(list)
14     for i in range(scores.shape[0]):
15         id2score[index[i]].append(scores[i])
16
17     # 3. 计算每组的均值和标准差
18     id2mean, id2std = {}, {}
19     for idx in id2score:
20         assert len(id2score[idx]) > 1, "GRPO needs rollout.n > 1."
21         id2mean[idx] = torch.mean(torch.tensor(id2score[idx]))
22         id2std[idx] = torch.std(torch.tensor(id2score[idx]))
23
24     # 4. 组内归一化
25     for i in range(scores.shape[0]):
26         scores[i] = (scores[i] - id2mean[index[i]]) / (id2std[index[i]] + eps)
27
28     # 5. 将归一化后的分数广播到所有 token (outcome reward 是序列级的)
29     returns = scores.unsqueeze(-1) * response_mask # (bs, response_len)
```

```
30
31     return returns, returns
```

公式对应：

GRPO Advantage：

$$\hat{A}_i = \frac{R_i - \mu_g}{\sigma_g + \epsilon}$$

RLOO

代码 (`verl/trainer/core_algos.py`) :

代码块

```
1  @register_adv_estimator(AdvantageEstimator.RLOO)
2  def compute_rloo_outcome_advantage(
3      token_level_rewards: torch.Tensor,
4      response_mask: torch.Tensor,
5      index: torch.Tensor,
6      **kwargs,
7  ) -> tuple[torch.Tensor, torch.Tensor]:
8      scores = token_level_rewards.sum(dim=-1) # (bs,)
9
10     # 1. 按 index 分组
11     id2score = defaultdict(list)
12     for i in range(scores.shape[0]):
13         id2score[index[i]].append(scores[i])
14
15     # 2. 计算每组的总和
16     id2sum = {}
17     for idx in id2score:
18         id2sum[idx] = torch.sum(torch.tensor(id2score[idx]))
19
20     # 3. Leave-one-out baseline
21     for i in range(scores.shape[0]):
22         sample_num = len(id2score[index[i]])
23         assert sample_num > 1, "RLOO needs rollout.n > 1."
24         # baseline = (总和 - 当前分数) / (n - 1)
25         baseline = (id2sum[index[i]] - scores[i]) / (sample_num - 1)
26         scores[i] = scores[i] - baseline
27
28     returns = scores.unsqueeze(-1) * response_mask
29     return returns, returns
```

公式对应：

RLOO Advantage:

$$\hat{A}_i = R_i - \frac{1}{n-1} \sum_{j \neq i} R_j$$

即：用其他 response 的平均分作为 baseline，减掉自己就得到 advantage。

更新 Critic

代码：

代码块

```
1  if self.use_critic:
2      critic_output = self.critic_wg.update_critic(batch)
```

`update_critic()` 的实现（在 `verl/workers/fsdp_workers.py`）：

代码块

```
1  @register(dispatch_mode=Dispatch.DP_COMPUTE_PROTO)
2  def update_critic(self, data: DataProto):
3      self._process_multi_modal_inputs(data)
4      data = data.to(torch.cuda.current_device())
5
6      if self._use_param_offload:
7          load_fsdp_model(self.fsdp_module)
8      if self._use_optimizer_offload:
9          load_fsdp_optimizer(optimizer=self.optimizer)
10
11     with self.ulysses_sharding_manager:
12         data = self.ulysses_sharding_manager.preprocess_data(data=data)
13         metrics = self.critic.update_value_function(data=data)
14         metrics = self.ulysses_sharding_manager.postprocess_data(metrics)
15
16     if self._use_param_offload:
17         offload_fsdp_model(self.fsdp_module)
18     if self._use_optimizer_offload:
19         offload_fsdp_optimizer(optimizer=self.optimizer)
20
21     return metrics
```

Critic 的更新逻辑（在 `verl/workers/critic/dp_critic.py`）：

代码块

```
1  def update_critic(self, data: DataProto) -> dict:
```

```

2     self.critic_module.train()
3
4     # 提取字段
5     select_keys = ["input_ids", "attention_mask", "position_ids", "values",
6 "returns", "response_mask"]
7     non_tensor_select_keys = ["multi_modal_inputs"]
8     data = data.select(select_keys, non_tensor_select_keys)
9
10    # 切分 mini-batch
11    mini_batches = data.split(self.config.global_batch_size_per_device)
12
13    for _ in range(self.config.ppo_epochs): # 通常是 1
14        for mini_batch in mini_batches:
15            # 切分 micro-batch (梯度累积)
16            micro_batches =
17            mini_batch.split(self.config.micro_batch_size_per_device)
18            total_response_tokens =
19            mini_batch.batch["response_mask"].sum().item()
20
21            for micro_batch in micro_batches:
22                model_inputs = {**micro_batch.batch,
23 **micro_batch.non_tensor_batch}
24
25                # 1. 前向计算 value
26                vpreds = self._forward_micro_batch(model_inputs) # (bs,
27 response_len)
28
29                # 2. 提取字段
30                old_values = micro_batch.batch["values"]
31                returns = micro_batch.batch["returns"]
32                response_mask = micro_batch.batch["response_mask"]
33
34                # 3. 计算 value loss
35                vf_loss, vf_metrics = compute_value_loss(
36                    vpreds=vpreds,
37                    returns=returns,
38                    values=old_values,
39                    response_mask=response_mask,
40                    cliprange_value=self.config.cliprange_value,
41                    loss_avg_mode=self.config.loss_avg_mode,
42                )
43
44                # 4. 反向传播 (归一化后的 loss)
45                loss = vf_loss * torch.sum(response_mask) * self.world_size /
46                total_response_tokens
47                loss.backward()
48

```

```

43         # 5. 梯度裁剪和优化器更新
44         grad_norm =
self.critic_module.clip_grad_norm_(self.config.max_grad_norm)
45         if not torch.isfinite(grad_norm):
46             print("Gradient norm is not finite. Skip update.")
47         else:
48             self.critic_optimizer.step()
49             self.critic_optimizer.zero_grad()
50
51     return vf_metrics

```

Value Loss 的计算 (在 `verl/trainer/core_algos.py`) :

代码块

```

1  def compute_value_loss(
2      vpreds: torch.Tensor,          # 当前 Critic 的预测 (bs, response_len)
3      returns: torch.Tensor,         # GAE 计算的 target (bs, response_len)
4      values: torch.Tensor,          # 旧的 value (bs, response_len)
5      response_mask: torch.Tensor,
6      cliprange_value: float,        # clip 范围, 默认 0.2
7      loss_avg_mode: str,
8  ) -> tuple[torch.Tensor, dict]:
9      # 1. Clip 预测的 value (防止更新太大)
10     vpredclipped = torch.clamp(vpreds, values - cliprange_value, values +
cliprange_value)
11
12     # 2. 计算两种 loss
13     vf_loss1 = torch.square(vpreds - returns)          # 未 clip 的 loss
14     vf_loss2 = torch.square(vpredclipped - returns)    # clip 后的 loss
15
16     # 3. 取最大值 (保守更新)
17     clipped_vf_losses = torch.max(vf_loss1, vf_loss2)
18
19     # 4. 平均
20     vf_loss = 0.5 * average_loss(clipped_vf_losses, response_mask,
mode=loss_avg_mode)
21
22     # 5. 指标
23     metrics = {
24         "vf_clipfrac": VF.masked_mean((vf_loss1 < vf_loss2).float(),
response_mask).item(),
25         "vpred_mean": VF.masked_mean(vpreds, response_mask).item(),
26     }
27     return vf_loss, metrics

```

公式对应：

Value Loss (带 clip)

$$L^{VF} = \frac{1}{2} \max [(V_{\phi}(s_t) - R_t)^2, (\text{clip}(V_{\phi}(s_t), V_{old}(s_t) - \epsilon, V_{old}(s_t) + \epsilon) - R_t)^2]$$

Clip 的作用：防止 Critic 更新太激进，保持稳定性。

更新 Actor

代码：

代码块

```
1  if self.config.trainer.critic_warmup <= self.global_step:
2      actor_output = self.actor_rollout_ref_wg.update_actor(batch)
```

`update_actor()` 的实现（在 `verl/workers/fsdp_workers.py`）：

代码块

```
1  @register(dispatch_mode=Dispatch.DP_COMPUTE_PROTO)
2  def update_actor(self, data: DataProto):
3      self._process_multi_modal_inputs(data)
4      data = data.to(torch.cuda.current_device())
5
6      if self._use_param_offload:
7          load_fsdp_model(self.fsdp_module)
8      if self._use_optimizer_offload:
9          load_fsdp_optimizer(optimizer=self.optimizer)
10
11     with self.ulysses_sharding_manager:
12         data = self.ulysses_sharding_manager.preprocess_data(data=data)
13         metrics = self.actor.update_policy(data=data)
14         metrics = self.ulysses_sharding_manager.postprocess_data(metrics)
15
16     if self._use_param_offload:
17         offload_fsdp_model(self.fsdp_module)
18     if self._use_optimizer_offload:
19         offload_fsdp_optimizer(optimizer=self.optimizer)
20
21     return metrics
```

Actor 的更新逻辑（在 `verl/workers/actor/dp_actor.py`）：

代码块

```

1  def update_policy(self, data: DataProto) -> dict:
2      self.actor_module.train()
3
4      temperature = data.meta_info["temperature"]
5
6      # 提取字段
7      select_keys = [
8          "input_ids", "attention_mask", "position_ids", "responses",
9          "old_log_probs", "advantages", "response_mask"
10     ]
11     if self.config.use_kl_loss:
12         select_keys.append("ref_log_probs")
13     non_tensor_select_keys = ["multi_modal_inputs"]
14
15     data = data.select(select_keys, non_tensor_select_keys)
16
17     # 切分 mini-batch
18     mini_batches = data.split(self.config.global_batch_size_per_device)
19
20     for _ in range(self.config.ppo_epochs): # 外层循环, 通常是 1
21         for mini_batch in mini_batches:
22             # 切分 micro-batch (梯度累积)
23             micro_batches =
24             mini_batch.split(self.config.micro_batch_size_per_device)
25             total_response_tokens =
26             mini_batch.batch["response_mask"].sum().item()
27
28             for micro_batch in micro_batches:
29                 model_inputs = {**micro_batch.batch,
30                 **micro_batch.non_tensor_batch}
31
32                 # 1. 前向计算 log_probs (重新计算, 不用缓存的 old_log_probs)
33                 log_probs = self._forward_micro_batch(model_inputs,
34                 temperature=temperature)
35
36                 # 2. 提取字段
37                 old_log_probs = micro_batch.batch["old_log_probs"]
38                 advantages = micro_batch.batch["advantages"]
39                 response_mask = micro_batch.batch["response_mask"]
40
41                 # 3. 计算 policy loss
42                 pg_loss, pg_metrics = compute_policy_loss(
43                     old_log_probs=old_log_probs,
44                     log_probs=log_probs,
45                     advantages=advantages,
46                     response_mask=response_mask,
47                     clip_ratio_low=self.config.clip_ratio_low,

```

```

44         clip_ratio_high=self.config.clip_ratio_high,
45         clip_ratio_dual=self.config.clip_ratio_dual,
46         loss_type=self.config.loss_type,
47         loss_avg_mode=self.config.loss_avg_mode,
48     )
49
50     # 4. 如果使用 KL loss (而不是 KL penalty)
51     if self.config.use_kl_loss and "ref_log_probs" in model_inputs:
52         ref_log_probs = micro_batch.batch["ref_log_probs"]
53         kld = compute_kl(
54             log_probs=log_probs,
55             ref_log_probs=ref_log_probs,
56             kl_penalty=self.config.kl_penalty,
57         )
58         kl_loss = average_loss(kld, response_mask,
mode=self.config.loss_avg_mode)
59         loss = pg_loss + kl_loss * self.config.kl_coef
60     else:
61         loss = pg_loss
62
63     # 5. 归一化 loss (按有效 token 数)
64     loss = loss * torch.sum(response_mask) * self.world_size /
total_response_tokens
65
66     # 6. 反向传播
67     loss.backward()
68
69     # 7. 梯度裁剪和优化器更新
70     grad_norm = self._optimizer_step()
71
72     # 8. 学习率调度
73     if self.lr_scheduler is not None:
74         self.lr_scheduler.step()
75
76     return pg_metrics

```

Policy Loss 的计算 (在 `verl/trainer/core_algos.py`) :

代码块

```

1  def compute_policy_loss(
2      old_log_probs: torch.Tensor,      # (bs, response_len)
3      log_probs: torch.Tensor,          # (bs, response_len)
4      advantages: torch.Tensor,        # (bs, response_len)
5      response_mask: torch.Tensor,      # (bs, response_len)
6      clip_ratio_low: float,            # PPO clip 下界, 默认 0.2

```



```

7     clip_ratio_high: float,          # PPO clip 上界, 默认 0.2
8     clip_ratio_dual: float,          # Dual clip, 默认 4.0
9     loss_type: str,                  # "default" | "gspo" | "cispo"
10    loss_avg_mode: str,               # "token" | "seq"
11    **kwargs,
12 ) -> tuple[torch.Tensor, dict]:
13     # 1. 计算 importance ratio 的对数
14     negative_approx_kl = log_probs - old_log_probs #  $\log(\pi_{new} / \pi_{old})$ 
15
16     # 2. 根据 loss_type 调整 importance ratio
17     if loss_type in ["gspo", "gspo_token"]:
18         # GSP0: 使用序列级的 importance ratio
19         negative_approx_kl_in_seq = VF.masked_mean(negative_approx_kl,
20 response_mask, dim=-1)
21         if loss_type == "gspo_token":
22             # 序列级 ratio + token 级调整
23             log_importance_ratio =
24 negative_approx_kl_in_seq.detach().unsqueeze(-1) + log_probs -
25 log_probs.detach()
26         else:
27             log_importance_ratio = negative_approx_kl_in_seq * response_mask
28         else:
29             # 标准 PPO: token 级 importance ratio
30             log_importance_ratio = negative_approx_kl
31
32     # 3. Clip before exp (防止 NaN)
33     ratio = torch.exp(torch.clamp(log_importance_ratio, -20.0, 20.0))
34     clipped_ratio = torch.exp(
35         torch.clamp(log_importance_ratio, np.log(1.0 - clip_ratio_low),
36 np.log(1.0 + clip_ratio_high))
37 )
38
39     # 4. 计算三种 loss
40     pg_loss = -advantages * ratio # 原始 loss
41     pg_loss2 = -advantages * clipped_ratio # Clip 后的 loss
42     pg_loss3 = -advantages * clip_ratio_dual # Dual clip (对负
43 advantage)
44
45     # 5. 根据 advantage 的符号选择 loss
46     clipped_pg_loss_higher = torch.max(pg_loss, pg_loss2) # 如果 pg_loss <
47 pg_loss2, 用 pg_loss2
48     clipped_pg_loss_lower = torch.min(clipped_pg_loss_higher, pg_loss3) # 如果
49 pg_loss_higher > pg_loss3, 用 pg_loss3
50
51     # 当 advantage > 0 时, 用 clipped_pg_loss_higher (标准 PPO clip)
52     # 当 advantage < 0 时, 用 clipped_pg_loss_lower (额外用 dual clip 保护)

```

```

46     final_pg_loss = torch.where(advantages < 0, clipped_pg_loss_lower,
47     clipped_pg_loss_higher)
48     # 6. 平均
49     final_pg_loss = average_loss(final_pg_loss, response_mask,
50     mode=loss_avg_mode)
51     # 7. 指标
52     metrics = {
53         "ppo_kl": -negative_approx_kl,
54         "entropy_loss": average_loss(-log_probs, response_mask,
55         mode=loss_avg_mode),
56         "pg_clipfrac_higher": (pg_loss < pg_loss2).float(),
57         "pg_clipfrac_lower": ((clipped_pg_loss_higher > pg_loss3).float() *
58         (advantages < 0).float()),
59     }
60     metrics = {k: VF.masked_mean(v, response_mask).detach().item() for k, v in
61     metrics.items()}
62     return final_pg_loss, metrics

```

公式对应：

Importance Ratio：

$$r_t = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{old}(a_t|s_t)} = \exp(\log \pi_{\theta} - \log \pi_{old})$$

PPO Clipped Objective（标准）：

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon_{low}, 1 + \epsilon_{high}) \hat{A}_t) \right]$$

DAPO（非对称 clip）：

- 当 $\hat{A}_t > 0$ 时（好动作），允许 ratio 更大的变化（ ϵ_{high} 可以 $> \epsilon_{low}$ ）
- 当 $\hat{A}_t < 0$ 时（差动作），额外用 dual clip 保护（防止过度惩罚）

Dual Clip（对负 advantage）：

$$L^{dual}(\theta) = \max(\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t, c \cdot \hat{A}_t)$$

最终的 Policy Loss：

$$L^{policy} = -\mathbb{E}_t[L^{CLIP}(\theta)]$$

注意前面有负号，因为我们要**最大化** objective，而优化器是做**最小化**。

GSPO： 详见 [📖居丽叶LLM体系知识搭建](#)

分布式训练（FSDP）

接下来看分布式训练框架FSDP的代码，负责协调 GPU 上执行所有模型相关的计算，包括模型初始化、前向推理、梯度更新、rollout 生成。 `verl/workers/fsdp_workers.py`。

初始化时根据 `role` 参数决定自己需要承担哪些角色

代码块

```
1  class FSDPWorker(Worker):
2      def __init__(
3          self,
4          config: WorkerConfig,
5          role: Literal["actor", "critic", "rollout", "ref", "actor_rollout",
6              "actor_rollout_ref"],
7      ):
8          super().__init__()
9          self.config = config
10         self.role = role
11
12         # 根据 role 决定需要初始化哪些组件
13         self._has_actor = self.role in ["actor", "actor_rollout",
14             "actor_rollout_ref"]
15         self._has_critic = self.role == "critic"
16         self._has_rollout = self.role in ["rollout", "actor_rollout",
17             "actor_rollout_ref"]
18         self._has_ref = self.role in ["ref", "actor_rollout_ref"]
19
20         if self.config.actor.disable_kl:
21             self._has_ref = False # 如果禁用 KL，就不需要参考策略
22
23         # 判断是否需要 CPU offload
24         self._use_param_offload = False
25         self._use_optimizer_offload = False
26         self._use_ref_param_offload = False
27         if self._has_actor:
28             self._use_param_offload = self.config.actor.offload.offload_params
29             self._use_optimizer_offload =
30             self.config.actor.offload.offload_optimizer
31             self._init_dist_mesh(self.config.actor, "actor")
```

Device mesh 定义了 GPU 之间的拓扑关系，支持一维（纯数据并行）和二维（数据并行 + 序列并行）：

- **一维 mesh** (`world_size,`)：所有 GPU 用于数据并行，每个 GPU 持有完整模型的一个分片
- **二维 mesh (DDP, FSDP)**：比如 16 GPU 分成 (2, 8)，2 个数据并行组，每组内 8 卡做模型分片
- **Ulysses SP mesh (DP, SP)**：序列并行，将长序列切分到多个 GPU 并行处理

```

代码块
def _init_dist_mesh(self, config: Union[ActorConfig, CriticConfig], role:
    Literal["actor", "critic"]):
2     world_size = dist.get_world_size()
3
4     # 创建主 device mesh
5     fsdp_size = config.fsdps.fsdps_size
6     if fsdp_size <= 0 or fsdp_size >= world_size:
7         # 一维 mesh: 纯 FSDP (数据并行)
8         self.device_mesh = init_device_mesh("cuda", mesh_shape=(world_size,),
        mesh_dim_names=("fsdp",))
9     else:
10        # 二维 mesh: HSDP (Hybrid Sharded Data Parallel)
11        # 第一维是 DDP (数据并行), 第二维是 FSDP (模型分片)
12        self.device_mesh = init_device_mesh(
13            "cuda", mesh_shape=(world_size // fsdp_size, fsdp_size),
        mesh_dim_names=("ddp", "fsdp")
14        )
15
16    # 创建 Ulysses 序列并行 device mesh (如果启用)
17    if config.ulysses_size > 1:
18        # 二维 mesh: (DP, SP)
19        # DP = 数据并行维度, SP = 序列并行维度
20        self.ulysses_device_mesh = init_device_mesh(
21            "cuda",
22            mesh_shape=(world_size // config.ulysses_size,
        config.ulysses_size),
23            mesh_dim_names=("dp", "sp"),
24        )
25    else:
26        self.ulysses_device_mesh = None

```

`init_model()` 按需构建模型。

代码块

```

1  @register(dispatch_mode=Dispatch.ONE_TO_ALL)
2  def init_model(self):
3      # 1. 初始化 Critic 模型 (如果需要)
4      if self._has_critic:
5          self._build_model_optimizer(
6              model_config=self.config.critic.model,
7              fsdp_config=self.config.critic.fsdps,
8              optim_config=self.config.critic.optim,
9              padding_free=self.config.critic.padding_free,
10             role="critic",
11         )

```

```

12
13     # 2. 初始化 Actor 模型 (训练用)
14     if self._has_actor:
15         self._build_model_optimizer(
16             model_config=self.config.actor.model,
17             fsdp_config=self.config.actor.fsdp,
18             optim_config=self.config.actor.optim,
19             padding_free=self.config.actor.padding_free,
20             role="actor",
21         )
22
23     # 3. 初始化 Ref 模型 (参考策略, 用于计算 KL 散度)
24     if self._has_ref:
25         self._build_model_optimizer(
26             model_config=self.config.actor.model,
27             fsdp_config=self.config.ref.fsdp,
28             optim_config=None, # Ref 不需要优化器
29             padding_free=self.config.ref.padding_free,
30             role="ref",
31         )
32
33     # 4. 包装 Actor 为 PPOActor (带 forward 和 update 逻辑)
34     if self._has_actor:
35         from .actor.dp_actor import DataParallelPPOActor
36         self.actor = DataParallelPPOActor(
37             config=self.config.actor,
38             actor_module=self.fsdp_module,
39             actor_optimizer=self.optimizer,
40         )
41
42     # 5. 初始化 vLLM Rollout 引擎 (生成 response 用)
43     if self._has_rollout:
44         self._build_rollout()

```

`_build_model_optimizer()` 是 FSDP 初始化的核心, 包含以下步骤:

1. 加载模型和配置

代码块

```

1 def _build_model_optimizer(
2     self,
3     model_config: ModelConfig,
4     fsdp_config: FSDPConfig,
5     optim_config: Optional[OptimConfig],
6     padding_free: bool,
7     role: Literal["actor", "critic", "ref"],

```

```

8  ) -> None:
9      if role != "ref": # ref 复用 actor 的 tokenizer
10         # 加载 tokenizer 和 processor (VLM 需要 processor 处理图像)
11         self.tokenizer = get_tokenizer(
12             model_config.tokenizer_path,
13             trust_remote_code=model_config.trust_remote_code,
14             use_fast=True,
15         )
16         self.processor = get_processor(
17             model_config.tokenizer_path,
18             trust_remote_code=model_config.trust_remote_code,
19             use_fast=True,
20         )
21         # 加载模型配置
22         self.model_config = AutoConfig.from_pretrained(
23             model_config.model_path,
24             trust_remote_code=model_config.trust_remote_code,
25             bos_token_id=self.tokenizer.bos_token_id,
26             eos_token_id=self.tokenizer.eos_token_id,
27             pad_token_id=self.tokenizer.pad_token_id,
28             **model_config.override_config,
29         )

```

2. 选择模型类和加载方式。

- 传统方式：所有 rank 都从磁盘加载完整模型权重 → 浪费时间和内存
- rank0_init: 只有 rank 0 加载权重，其他 rank 创建空模型 → FSDP 初始化时通过 `sync_module_states=True` 从 rank 0 广播权重

代码块

```

1  if role == "critic":
2      AutoClass = AutoModelForTokenClassification # Critic 是 token 分类任务
3  elif type(self.model_config) in
4      AutoModelForImageTextToText._model_mapping.keys():
5      AutoClass = AutoModelForImageTextToText # VLM 模型
6  else:
7      AutoClass = AutoModelForCausalLM # 标准 LLM
8
9  # 加载模型权重
10 if (not fsdp_config.enable_rank0_init) or
11     self.device_mesh.get_local_rank("fsdp") == 0:
12     # 正常加载：所有 rank 都加载完整模型 (或 rank 0 加载后同步)
13     model = AutoClass.from_pretrained(
14         model_config.model_path,
15         config=self.model_config,

```

```

14         torch_dtype=torch_dtype,
15         attn_implementation="flash_attention_2", # 使用 Flash Attention 2
16         device_map="cpu" if fsdp_config.enable_rank0_init else "cuda",
17         low_cpu_mem_usage=True,
18         trust_remote_code=model_config.trust_remote_code,
19     )
20 else:
21     # rank0_init 模式: 其他 rank 只创建空壳, 不加载权重 (节省内存和时间)
22     with no_init_weights(), init_empty_weights():
23         model = AutoClass.from_config(
24             self.model_config,
25             torch_dtype=torch_dtype,
26             attn_implementation="flash_attention_2",
27             trust_remote_code=model_config.trust_remote_code,
28         )

```

3. 配置梯度检查点, ref模型冻结参数, 冻结视觉编码器

代码块

```

1  model = cast(PreTrainedModel, model)
2  model.tie_weights() # 绑定输入输出 embedding (避免同步问题)
3  model = model.to(torch_dtype)
4
5  # 启用 Gradient Checkpointing (用计算换显存)
6  if model_config.enable_gradient_checkpointing:
7      model.gradient_checkpointing_enable(gradient_checkpointing_kwargs=
8          {"use_reentrant": False})
9
10 # Ref 模型冻结所有参数
11 if role == "ref":
12     model.requires_grad_(False)
13
14 # 冻结视觉编码器 (VLM 训练中常用)
15 if model_config.freeze_vision_tower:
16     if hasattr(model, "model") and hasattr(model.model, "visual"):
17         model.model.visual.requires_grad_(False)
18         fsdp_config.use_orig_params = True # 冻结部分参数时必须用 use_orig_params
19         self.print_rank0("Vision tower is set to not trainable.")

```

4. 配置混合精度

- `param_dtype`: 模型参数的存储精度 (通常 BF16 或 FP16)
- `reduce_dtype`: 梯度 All-Reduce 通信时的精度 (通常和 `param_dtype` 一致)

- `buffer_dtype` : LayerNorm 等模块的统计量精度（通常保持 FP32 以确保数值稳定）

代码块

```
1 mixed_precision = MixedPrecision(  
2     param_dtype=PrecisionType.to_dtype(fsdp_config.mp_param_dtype),      # 参数存  
   储精度  
3     reduce_dtype=PrecisionType.to_dtype(fsdp_config.mp_reduce_dtype),    # 梯度通  
   信精度  
4     buffer_dtype=PrecisionType.to_dtype(fsdp_config.mp_buffer_dtype),    # buffer  
   (如 LN) 精度  
5 )
```

5. 配置 Auto Wrap Policy（`verl/utils/fsdp_utils.py`）。FSDP 将模型分成多个单元，每个单元的参数作为一个整体分片和通信：`transformer_auto_wrap_policy` 自动识别模型的 Transformer 层类（如 `Qwen2VLDecoderLayer`、`LlamaDecoderLayer`），将每层包装为一个 FSDP 单元。

代码块

```
1 auto_wrap_policy = get_fsdp_wrap_policy(model)  
2 def get_fsdp_wrap_policy(model: PreTrainedModel):  
3     """获取 FSDP 的自动包装策略  
4  
5     根据模型的 _no_split_modules 属性，自动找到 Transformer 层的类名，  
6     然后用 transformer_auto_wrap_policy 将每个 Transformer 层包装为一个 FSDP 单元  
7     """  
8     transformer_cls_to_wrap = set()  
9     for module in model._no_split_modules:  
10         transformer_cls = get_module_class_from_name(model, module)  
11         if transformer_cls is None:  
12             raise Exception(f"Cannot find {module} in pretrained model.")  
13         else:  
14             transformer_cls_to_wrap.add(transformer_cls)  
15  
16     return partial(transformer_auto_wrap_policy,  
                    transformer_layer_cls=transformer_cls_to_wrap)
```

6. 分片策略设置

代码块

```
1 if self.device_mesh.ndim == 2:  
2     # 二维 mesh (HSDP)  
3     if fsdp_config.enable_full_shard:
```



```

4         sharding_strategy = ShardingStrategy.HYBRID_SHARD # 完全分片版本
5     else:
6         sharding_strategy = ShardingStrategy._HYBRID_SHARD_ZERO2 # ZeRO-2 版本
7     else:
8         # 一维 mesh (纯 FSDP)
9         if fsdp_config.enable_full_shard:
10            sharding_strategy = ShardingStrategy.FULL_SHARD # 参数+梯度+优化器全分片
11        else:
12            sharding_strategy = ShardingStrategy.SHARD_GRAD_OP # 仅梯度+优化器分片
13            (ZeRO-2)
14    # 配置 CPU Offload
15    if fsdp_config.enable_cpu_offload:
16        cpu_offload = CPUOffload(offload_params=True)
17    else:
18        cpu_offload = None

```

HYBRID_SHARD 原理（多节点训练的最佳选择）：

假设 2 节点 × 8 GPU，模型 20B 参数：

- **FULL_SHARD**：16 GPU 完全分片，每卡 1.25B 参数
 -  前向时需要向其他 15 个 GPU All-Gather 参数，跨节点通信慢
- **HYBRID_SHARD**：节点内 FULL_SHARD，节点间 DDP
 -  前向只在节点内 All-Gather（NVLink 快）
 -  反向只在节点间同步梯度（一次通信）

7. 包装成 FSDP 模块

- **sync_module_states=True**：
 - 在初始化时，rank 0 将参数广播到其他 rank
 - 确保所有 GPU 的初始权重完全一致（避免从不同 checkpoint 分片加载导致不一致）
- **use_orig_params**：
 - False（默认）：FSDP 将所有参数 flatten 成一个大 Tensor（FlatParameter），节省显存但难以调试
 - True：保持原始参数结构（如 `model.layers[0].self_attn.q_proj.weight`），便于调试和兼容某些优化器
 - **冻结部分参数时必须设为 True**（否则 FSDP 无法区分可训练和冻结参数）
- **forward_prefetch=False**：
 - True：提前预取下一层的参数（计算和通信重叠，提速但占显存）

- False: 当前层计算完再释放参数，不提前预取（节省显存，但可能慢一点）

代码块

```
1  if fsdp_config.enable_rank0_init:
2      # rank0_init 模式: 需要从 rank 0 同步权重
3      sync_module_states = True
4      param_init_fn = get_init_fn(model, device="cuda") if self.rank != 0 else
      None
5  else:
6      sync_module_states = False
7      param_init_fn = None
8
9  fsdp_module = FSDP(
10     model,
11     sharding_strategy=sharding_strategy,
12     cpu_offload=cpu_offload,
13     auto_wrap_policy=auto_wrap_policy,
14     mixed_precision=mixed_precision,
15     param_init_fn=param_init_fn, # 非 rank 0 的参数初始化函数
16     device_id=torch.cuda.current_device(),
17     sync_module_states=sync_module_states, # 是否从 rank 0 同步权重
18     forward_prefetch=False, # 不提前预取参数（节省显存）
19     use_orig_params=fsdp_config.use_orig_params, # 是否保持原始参数结构
20     device_mesh=self.device_mesh,
21 )
```

8. 创建优化器和学习率调度器

代码块

```
1  if role in ["actor", "critic"]:
2      self.fsdp_module = fsdp_module
3
4      # 创建优化器
5      if optim_config.strategy == "adamw":
6          self.optimizer = torch.optim.AdamW(
7              filter(lambda p: p.requires_grad, self.fsdp_module.parameters()),
8              lr=optim_config.lr,
9              betas=optim_config.betas,
10             weight_decay=optim_config.weight_decay,
11             fused=True, # 使用 CUDA fused kernel (更快)
12         )
13     elif optim_config.strategy == "adamw_bf16":
14         self.optimizer = AnyPrecisionAdamW( # 支持 BF16 优化器状态
15             filter(lambda p: p.requires_grad, self.fsdp_module.parameters()),
16             lr=optim_config.lr,
```

```

17         betas=optim_config.betas,
18         weight_decay=optim_config.weight_decay,
19     )
20
21     # 创建学习率调度器
22     num_warmup_steps = optim_config.lr_warmup_steps or
23     int(optim_config.lr_warmup_ratio * optim_config.training_steps)
24
25     if optim_config.lr_scheduler_type == "constant":
26         self.lr_scheduler = get_constant_schedule_with_warmup(
27             optimizer=self.optimizer, num_warmup_steps=num_warmup_steps
28         )
29     elif optim_config.lr_scheduler_type == "cosine":
30         self.lr_scheduler = get_cosine_schedule_with_warmup(
31             optimizer=self.optimizer,
32             num_warmup_steps=num_warmup_steps,
33             num_training_steps=optim_config.training_steps,
34             min_lr_ratio=optim_config.min_lr_ratio,
35             num_cycles=0.5,
36         )
37
38     # 如果启用了 CPU offload, 将模型和优化器卸载到 CPU
39     if self._use_param_offload:
40         offload_fsdp_model(self.fsdp_module)
41     if self._use_optimizer_offload:
42         offload_fsdp_optimizer(optimizer=self.optimizer)
43     else:
44         # Ref 模型不需要优化器
45         self.ref_fsdp_module = fsdp_module
46         if self._use_ref_param_offload:
47             offload_fsdp_model(self.ref_fsdp_module)

```

Rollout (VLLM)

负责用当前策略生成 response, 本项目用vllm推理框架

1. vLLMRollout 在初始化时创建 vLLM 引擎, 但不加载权重 (从 FSDP 同步)

(`verl/workers/rollout/vllm_rollout_spmd.py`)

- `load_format="dummy"`: 只创建模型结构 (空权重), 不从磁盘加载。权重会在推理前从 FSDP 模型同步过来
- `enable_sleep_mode=True` + `sleep(level=1)`: 初始化后立即释放 KV cache、权重等占用的显存, 让 FSDP 训练能使用这些显存
- `distributed_executor_backend="external_launcher"`: vLLM 使用外部已初始化的 `torch.distributed`, 而不是自己再初始化一遍

代码块

```
1  class vLLMRollout(BaseRollout):
2      def __init__(
3          self,
4          model_path: str,
5          config: RolloutConfig,
6          tokenizer: PreTrainedTokenizer,
7          processor: Optional[ProcessorMixin],
8      ):
9          super().__init__()
10         self.rank = int(os.getenv("RANK", "0"))
11         self.config = config
12         self.pad_token_id = tokenizer.pad_token_id
13         self.use_tqdm = (self.rank == 0) and (not config.disable_tqdm)
14
15         # VLM 特殊配置: 限制每个 prompt 的图片数量
16         engine_kwargs = {}
17         if processor is not None:
18             if config.limit_images:
19                 engine_kwargs["limit_mm_per_prompt"] = {"image":
config.limit_images}
20
21         # 创建 vLLM 引擎
22         self.inference_engine = LLM(
23             model=model_path,
24             skip_tokenizer_init=False,
25             trust_remote_code=config.trust_remote_code,
26             load_format="dummy", # 关键! 不加载权重, 后续从 FSDP 同步
27             dtype=PrecisionType.to_str(PrecisionType.to_dtype(config.dtype)),
28             seed=config.seed,
29             max_model_len=config.max_model_len or config.prompt_length +
config.response_length,
30             distributed_executor_backend="external_launcher", # 使用外部的
torch.distributed
31             tensor_parallel_size=config.tensor_parallel_size,
32             gpu_memory_utilization=config.gpu_memory_utilization, # 例如 0.9
(用 90% 显存)
33             max_num_batched_tokens=config.max_num_batched_tokens, # 限制 batch
内总 token 数
34             disable_log_stats=config.disable_log_stats,
35             enforce_eager=config.enforce_eager, # 是否禁用 CUDA graph (调试时用)
36             disable_custom_all_reduce=True,
37             enable_chunked_prefill=config.enable_chunked_prefill, # 分块
prefill (节省显存)
38             enable_sleep_mode=True, # 关键! 支持 sleep/wake 释放显存
39             **engine_kwargs,
```

```

40         )
41
42         # 关键! 初始化后立即 sleep, 释放显存给 FSDP 训练使用
43         self.inference_engine.sleep(level=1)
44
45         # 配置采样参数
46         sampling_kwargs = {
47             "max_tokens": config.response_length,
48             "detokenize": False, # 不解码成文本 (返回 token ids)
49             "logit_bias": _get_logit_bias(processor), # VLM: 禁止生成图片 token
50         }
51         default_sampling_params = SamplingParams()
52         for key in config.to_dict().keys():
53             if hasattr(default_sampling_params, key):
54                 sampling_kwargs[key] = getattr(config, key)
55
56         self.sampling_params = SamplingParams(**sampling_kwargs)

```

2. FSDP → vLLM 权重同步 (`verl/workers/sharding_manager/fsdp_vllm.py`)

代码块

```

1  def load_vllm_and_sync_weights(self):
2      """唤醒 vLLM 引擎并同步 FSDP 模型权重到 vLLM"""
3      # 1. 清空 PyTorch 显存缓存, 让 vLLM 的 CuMemAllocator 有空间分配
4      torch.cuda.empty_cache()
5      assert self.loaded is False, "vllm engine has already been loaded"
6      self.loaded = True
7
8      # 2. 唤醒 vLLM: 分配 KV cache、模型权重等显存
9      print_gpu_memory_usage("Before vllm wake up in sharding manager")
10     if "tags" in inspect.signature(self.inference_engine.wake_up).parameters:
11         # vLLM 0.7+ 支持分阶段唤醒: 先唤醒权重部分
12         self.inference_engine.wake_up(tags=["weights"])
13     else:
14         self.inference_engine.wake_up()
15
16     # 3. 同步权重: FSDP → vLLM
17     self._sync_weight_to_vllm()
18
19     # 4. 唤醒 KV cache 部分
20     if "tags" in inspect.signature(self.inference_engine.wake_up).parameters:
21         self.inference_engine.wake_up(tags=["kv_cache"])
22
23     # 5. 重置多模态缓存
24     # 避免 sleep 后缓存不一致导致 assert 失败

```

```

25     if hasattr(self.inference_engine, "reset_mm_cache"):
26         self.inference_engine.reset_mm_cache()
27
28     # 6. 设置推理时的随机种子 (保证 TP 组内所有卡生成一致)
29     if self.device_mesh is not None:
30         self.torch_random_states = torch.cuda.get_rng_state() # 保存训练用的
RNG 状态
31         torch.cuda.set_rng_state(self.gen_random_states) # 切换到推理用的
RNG 状态
32
33
34     def _sync_weight_to_vllm(self):
35         # 1. 如果启用了 CPU offload, 先将 FSDP 模型加载回 GPU
36         if self.use_param_offload:
37             load_fsdp_model(self.module)
38
39         # 2. 从 FSDP 模型提取权重 (All-Gather 所有分片)
40         actor_weights = get_model_state_dict(self.module) # 返回 {name: DTensor}
41
42         # 3. 处理 Transformers 版本差异 (key 命名转换)
43         actor_weights = self._rename_weight_keys(actor_weights,
self.module._fsdp_wrapped_module)
44         print_gpu_memory_usage("After gather model weights in sharding manager")
45
46         # 4. 将权重加载到 vLLM 模型
47         model =
self.inference_engine.llm_engine.model_executor.driver_worker.worker.model_runn
er.model
48         model.load_weights(self._make_weight_iterator(actor_weights))
49         # _make_weight_iterator: 如果是 DTensor, 调用 .full_tensor() 还原成普通 Tensor
50
51         # 5. 清理临时权重, 恢复 CPU offload
52         del actor_weights
53         if self.use_param_offload:
54             offload_fsdp_model(self.module)
55
56         torch.cuda.empty_cache()
57         print_gpu_memory_usage("After sync model weights in sharding manager")

```

3. 生成sequence, `generate_sequences()` 是 rollout 的核心入口, 负责数据预处理、调用 vLLM 生成、数据后处理 (`verl/workers/fsdp_workers.py`):

代码块

```

1 @register(dispatch_mode=Dispatch.DP_COMPUTE_PROTO)
2 def generate_sequences(self, prompts: DataProto):

```

```

3     assert self._has_rollout
4
5     # 1. 补充 EOS 和 PAD token id (从 generation_config 或 tokenizer 获取)
6     meta_info = {
7         "eos_token_id": self.generation_config.eos_token_id
8         if self.generation_config is not None
9         else self.tokenizer.eos_token_id,
10        "pad_token_id": self.generation_config.pad_token_id
11        if self.generation_config is not None
12        else self.tokenizer.pad_token_id,
13    }
14    prompts.meta_info.update(meta_info)
15
16    # 2. 预处理: TP 组内 All-Gather (让每个 TP rank 都有完整数据)
17    prompts = self.rollout_sharding_manager.preprocess_data(prompts)
18
19    # 3. 调用 vLLM 生成
20    output = self.rollout.generate_sequences(prompts=prompts)
21
22    # 4. 后处理: 提取当前 TP rank 对应的数据分片
23    output = self.rollout_sharding_manager.postprocess_data(output)
24
25    # 5. 返回到 CPU (下一步要送到 Actor 计算 log_probs)
26    output = output.to("cpu")
27    return output

```

vLLM 生成核心逻辑 (`verl/workers/rollout/vllm_rollout_spmd.py`)

代码块

```

1     @torch.no_grad()
2     def generate_sequences(self, prompts: DataProto) -> DataProto:
3         # 1. 提取输入数据
4         input_ids: torch.Tensor = prompts.batch["input_ids"] # (bs,
5         # prompt_length) 左填充
6         attention_mask: torch.Tensor = prompts.batch["attention_mask"]
7         position_ids: torch.Tensor = prompts.batch["position_ids"]
8         eos_token_id: int = prompts.meta_info["eos_token_id"]
9         batch_size = input_ids.size(0)
10
11        # 2. 提取原始 prompt ids 和多模态数据
12        non_tensor_batch = prompts.non_tensor_batch
13        batch_raw_prompt_ids = non_tensor_batch.pop("raw_prompt_ids") # 未填充的原始 token ids
14        batch_multi_modal_data = non_tensor_batch.pop("multi_modal_data", None) # 图片/视频数据

```

```

14
15     # 3. 构建 vLLM 输入格式
16     if batch_multi_modal_data is not None:
17         # VLM: 包含图片/视频
18         vllm_inputs = []
19         for raw_prompt_ids, multi_modal_data in zip(batch_raw_prompt_ids,
batch_multi_modal_data):
20             vllm_inputs.append(
21                 {
22                     "prompt_token_ids": list(raw_prompt_ids), # vLLM 要求 list
格式
23                     "multi_modal_data": _process_multi_modal_data(
24                         multi_modal_data,
25                         prompts.meta_info["min_pixels"],
26                         prompts.meta_info["max_pixels"],
27                         prompts.meta_info["video_fps"],
28                     ),
29                 }
30             )
31     else:
32         # 纯 LLM
33         vllm_inputs = [{"prompt_token_ids": list(raw_prompt_ids)} for
raw_prompt_ids in batch_raw_prompt_ids]
34
35     # 4. 调用 vLLM 生成 (支持临时修改采样参数)
36     with self.update_sampling_params(**prompts.meta_info):
37         completions: list[RequestOutput] = self.inference_engine.generate(
38             prompts=vllm_inputs, sampling_params=self.sampling_params,
use_tqdm=self.use_tqdm
39         )
40         # 提取 response token ids (每个 completion 可能有多个 output, 对应 n > 1)
41         response_ids = [output.token_ids for completion in completions for
output in completion.outputs]
42         # 右填充到固定长度
43         response_ids = VF.pad_2d_list_to_length(
44             response_ids, self.pad_token_id,
max_length=self.config.response_length
45         ).to(input_ids.device)
46
47     # 5. 如果 n > 1 (每个 prompt 生成多个 response) , 需要 repeat prompt 对齐
48     if self.sampling_params.n > 1:
49         batch_size = batch_size * self.sampling_params.n
50         input_ids = _repeat_interleave(input_ids, self.sampling_params.n)
51         attention_mask = _repeat_interleave(attention_mask,
self.sampling_params.n)
52         position_ids = _repeat_interleave(position_ids,
self.sampling_params.n)

```



```

53         if batch_multi_modal_data is not None:
54             batch_multi_modal_data =
_repeat_interleave(batch_multi_modal_data, self.sampling_params.n)
55
56         # 6. 拼接 prompt + response
57         sequence_ids = torch.cat([input_ids, response_ids], dim=-1)
58
59         # 7. 构建 response 的 position_ids
60         response_length = response_ids.size(1)
61         delta_position_id = torch.arange(1, response_length + 1,
device=position_ids.device)
62         delta_position_id = delta_position_id.view(1, -1).expand(batch_size, -1)
63
64         if position_ids.ndim == 3: # (batch_size, 4, seq_length)
65             delta_position_id = delta_position_id.view(batch_size, 1,
-1).expand(batch_size, position_ids.size(1), -1)
66
67         # prompt 的最后一个 position + [1, 2, 3, ..., response_length]
68         response_position_ids = position_ids[..., -1:] + delta_position_id
69         position_ids = torch.cat([position_ids, response_position_ids], dim=-1)
70
71         # 8. 构建 response_mask (标记有效 response token)
72         # attention_mask: [0,0,0,0,1,1,1,1 | 1,1,1,0,0,0,0,0]
73         #             ↑ prompt 左填充 | ↑ response 右填充
74         response_mask = VF.get_response_mask(
75             response_ids=response_ids, eos_token_id=eos_token_id,
dtype=attention_mask.dtype
76         )
77         attention_mask = torch.cat((attention_mask, response_mask), dim=-1)
78
79         # 9. 返回结果
80         batch = TensorDict(
81             {
82                 "prompts": input_ids,
83                 "responses": response_ids,
84                 "input_ids": sequence_ids, # 完整序列 (prompt + response)
85                 "attention_mask": attention_mask,
86                 "response_mask": response_mask, # 用于计算 loss 时只关注 response 部
分
87                 "position_ids": position_ids,
88             },
89             batch_size=batch_size,
90         )
91         if batch_multi_modal_data is not None:
92             non_tensor_batch = {"multi_modal_data": batch_multi_modal_data}
93         else:
94             non_tensor_batch = {}

```

```
95
96     return DataProto(batch=batch, non_tensor_batch=non_tensor_batch,
    meta_info=prompts.meta_info)
```

4. 释放 Rollout 引擎 (vLLM → FSDP 切换) (

verl/workers/sharding_manager/fsdp_vllm.py)

代码块

```
1  def offload_vllm(self):
2      """让 vLLM 引擎 sleep, 释放显存"""
3      assert self.loaded is True, "vllm engine has not been loaded"
4      self.loaded = False
5
6      # 1. 记录 sleep 前后的显存变化 (用于后续计算训练时的显存占用)
7      print_gpu_memory_usage("Before vllm offload in sharding manager")
8      free_bytes_before_sleep = torch.cuda.mem_get_info()[0]
9      self.inference_engine.sleep(level=1) # 释放 KV cache、权重等
10     free_bytes_after_sleep = torch.cuda.mem_get_info()[0]
11     self.freed_bytes = free_bytes_after_sleep - free_bytes_before_sleep
12     print_gpu_memory_usage("After vllm offload in sharding manager")
13
14     # 2. 切换回训练模式
15     self.module.train()
16     torch.cuda.empty_cache()
17
18     # 3. 恢复训练用的随机种子
19     if self.device_mesh is not None:
20         self.gen_random_states = torch.cuda.get_rng_state() # 保存推理用的 RNG
    状态
21         torch.cuda.set_rng_state(self.torch_random_states) # 恢复训练用的 RNG
    状态
```



为什么不直接用 vLLM 训练?

- vLLM 是推理引擎, 不支持梯度反传和优化器更新
- vLLM 的 PagedAttention 对训练不友好 (动态 KV cache 不适合反向传播)

为什么不直接用 FSDP 推理?

- FSDP 推理慢: 每次前向都要 All-Gather 参数, 通信开销大
- vLLM 有 PagedAttention、Continuous Batching 等优化, 吞吐量高 3-5 倍

开发部分

代码运行

1. 环境准备

代码块

```
1  conda create -n medix-swarm python=3.12 -y
2  conda activate medix-swarm
3  cd medix-agent-swarm
```

2. 安装依赖

代码块

```
1  pip install -r requirements.txt
```

3. 配置 API

创建 `../config.py`，mem0 apikey获取：<https://app.mem0.ai/dashboard/api-keys>

代码块

```
1  LLM_CONFIG = {
2      "api_key": "your-llm-api-key",
3      "model_name": "your-model-name",
4      "base_url": "https://api.openai.com/v1",
5      "temperature": 0.7,
6      "max_tokens": 8192,
7  }
8
9  MEM0_CONFIG = {"api_key": "m0-your-api-key-here"}
```

4. 初始化知识库

代码块

```
1  python knowledge/scripts/import_hardcoded_data.py
```

5. 开始使用

代码块

```
1 python main.py
```

代码解读

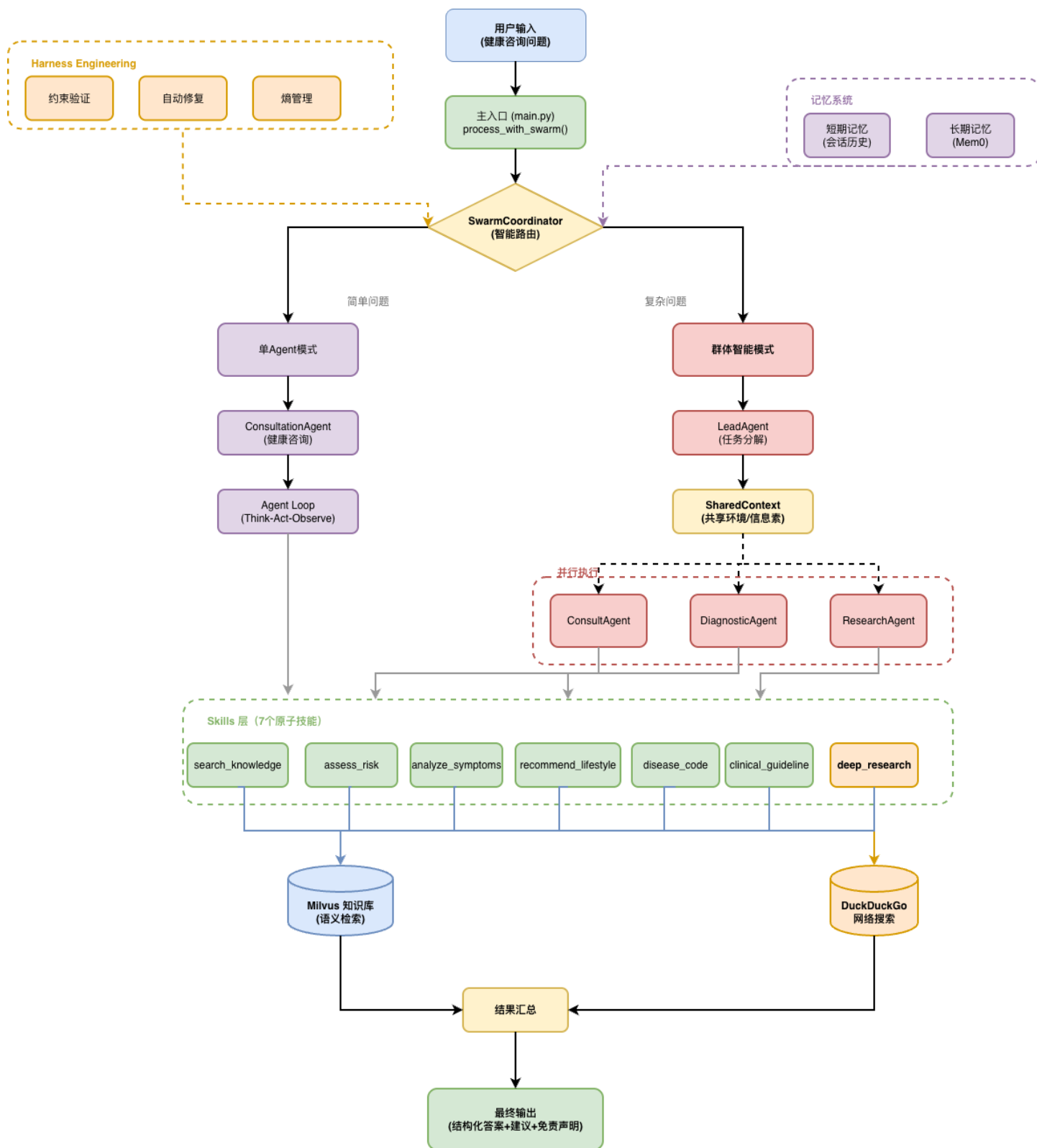
参考项目：<https://github.com/desplega-ai/agent-swarm>

MediX Agent Swarm 是一个基于 Skills-Agent 两层架构的多智能体协作医疗助手系统，整体流程为：

1. 用户输入：问题通过 main.py 入口进入系统
2. 智能路由：LeadAgent 将复杂问题拆解为多个子任务，简单问题→单Agent，复杂问题→Swarm
3. 并行执行：Worker Agents 自主认领任务，调用 Skills 并行处理
4. 结果汇总：LeadAgent 综合各 Agent 的结果生成最终答案
5. 记忆沉淀：短期记忆存储对话历史，长期记忆存储会话总结

系统核心特性：

- Skills-Agent 两层架构：Agent 自主选择 Skills
- Agent Loop：LLM 驱动的 Think-Act-Observe 工具调用循环
- Agent Swarm：去中心化群体智能，通过 SharedContext（信息素）间接通信，自主任务认领，并行执行
- 记忆系统：短期记忆（会话级对话历史）+ 长期记忆（跨会话，存储历史会话）
- Harness Engineering：约束验证 + 熵管理，保证系统安全和简洁



系统入口

main函数调用 SwarmCoordinator 处理用户问题

代码块

```

1  async def process_with_swarm(
2      question: str,
3      context: Optional[Dict[str, Any]] = None,
4      enable_swarm: bool = True,
5      session_id: Optional[str] = None

```

```

6 ) -> Dict[str, Any]:
7     """便捷函数：使用 Swarm 处理问题"""
8     coordinator = SwarmCoordinator(enable_swarm=enable_swarm)
9     return await coordinator.process(question, context, session_id=session_id)

```

智能路由

- 代码路径: `swarm/swarm_coordinator.py`
- 职责: 根据任务复杂度路由到单Agent或Swarm

SwarmCoordinator 在初始化时创建了所有 Worker Agents 和记忆管理器:

代码块

```

1 class SwarmCoordinator:
2     def __init__(
3         self,
4         llm_client: Optional[LLMClient] = None,
5         enable_swarm: bool = True,
6         complexity_threshold: float = 0.6
7     ):
8         # 初始化 Agent
9         self.lead_agent = LeadAgent(llm_client=self.llm_client)
10        self.consultation_agent = ConsultationAgent()
11        self.diagnostic_agent = DiagnosticAgent()
12        self.research_agent = ResearchAgent()
13
14        # Worker 池
15        self.worker_pool: List[Any] = [
16            self.consultation_agent,
17            self.diagnostic_agent,
18            self.research_agent
19        ]
20
21        # 记忆管理器
22        self.session_manager = SessionSummaryManager()
23        self.short_term_memory = ShortTermMemory(storage_type="memory")
24        self.long_term_memory = LongTermMemory()
25
26        # 将短期记忆注入到所有 Worker Agent 的 Loop
27        for worker in self.worker_pool:
28            if hasattr(worker, 'loop'):
29                worker.loop.short_term_memory = self.short_term_memory

```

路由规则:

- **1个子任务**: 单Agent直接处理
- **≥2个子任务 + Swarm开启**: Swarm 多Agent协作
- **0个子任务或Swarm关闭**: 降级到 ConsultationAgent

代码块

```
1  # Step 1: LeadAgent 分解任务
2  assessment = await self.lead_agent.assess_and_decompose(question,
3  enhanced_context)
4
5  # Step 2: 根据任务数量路由
6  if len(subtasks) == 1:
7      # 单任务 → 直接调用对应 Agent
8      task = subtasks[0]
9      agent_id = task.get("assigned_agent")
10     agent = self._get_agent_by_id(agent_id)
11
12     mode = "single_agent"
13     result = await agent.process({
14         'question': question,
15         'context': enhanced_context,
16         'session_id': session_id
17     })
18
19 elif len(subtasks) >= 2 and self.enable_swarm:
20     # 多任务 → 启动 Swarm
21     mode = "swarm"
22     result = await self._process_with_swarm(
23         question=question,
24         context=enhanced_context,
25         assessment=assessment,
26         session_id=session_id,
27         start_time=start_time
28     )
29
30 else:
31     # 0个任务或Swarm关闭 → 降级到 ConsultationAgent
32     if len(subtasks) == 0:
33         logger.warning("No subtasks generated, fallback to ConsultationAgent")
34         mode = "fallback"
35     else:
36         logger.info("Swarm disabled, fallback to ConsultationAgent")
37         mode = "disabled_swarm"
38
39     result = await self.consultation_agent.process({
```

```

40         'question': question,
41         'context': enhanced_context,
42         'session_id': session_id
43     })

```

任务分解与汇总

- 路径: `swarm/lead_agent.py`
- 职责: 分解复杂任务为子任务, 汇总 Worker 结果

1. **任务分解**: 将问题拆成多个独立的子任务

2. **Agent 分配**: 为每个子任务推荐最合适的 Agent

代码块

```

1  async def assess_and_decompose(
2      self,
3      question: str,
4      context: Optional[Dict[str, Any]] = None
5  ) -> Dict[str, Any]:
6      """评估问题复杂度并分解任务"""
7
8      # 构建提示词
9      prompt = f"""你是医疗助手的任务分解专家。分析用户问题，判断复杂度并分解任务...
10
11  用户问题: {question}
12
13  请以JSON格式返回:
14  {{
15      "subtasks": [
16          {{
17              "description": "具体任务描述",
18              "assigned_agent": "建议的Agent ID"
19          }}
20      ]
21  }}
22
23  Agent选择:
24  - consultation_agent: 通用健康咨询
25  - diagnostic_agent: 症状诊断和风险评估
26  - research_agent: 医学研究和证据检索
27  """
28
29      response = await self.llm_client.chat_with_tools(
30          messages=[{"role": "user", "content": prompt}],
31          tools=None, # 不需要工具调用

```



```

32         temperature=0.3
33     )
34
35     # 解析 JSON 响应
36     assessment = json.loads(response.content)
37     return assessment

```

对于开启了agent swarm的任务，LeadAgent 会创建具体的 Subtask 对象并发布到 SharedContext:

代码块

```

1  def create_subtasks(
2      self,
3      assessment: Dict[str, Any],
4      shared_context: 'SharedContext'
5  ) -> List['Subtask']:
6      """创建子任务并发布到 SharedContext"""
7
8      subtasks = []
9      for task_data in assessment.get("subtasks", []):
10         subtask = Subtask(
11             id=str(uuid.uuid4()),
12             type=task_data["type"],
13             description=task_data["description"],
14             assigned_to=task_data.get("assigned_agent"),
15             priority=task_data.get("priority", 2),
16             required_capabilities=task_data.get("required_capabilities", [])
17         )
18
19         # 发布到 SharedContext
20         shared_context.add_subtask(subtask)
21         subtasks.append(subtask)
22
23         logger.info(
24             f"Created subtask: {subtask.type} -> {subtask.assigned_to}"
25         )
26
27     return subtasks

```

等agent swarm执行结束后，LeadAgent 负责汇总结果:

代码块

```

1  async def synthesize_results(
2      self,
3      question: str,

```

```

4     shared_context: 'SharedContext',
5     timeout_occurred: bool = False
6 ) -> str:
7     """综合各 Agent 的结果生成最终答案"""
8
9     # 收集各 Agent 的贡献
10    agent_contributions = shared_context.agent_contributions
11
12    if not agent_contributions:
13        return "抱歉，系统未能完成您的问题分析。"
14
15    # 构建汇总提示词
16    contributions_text = []
17    for agent_id, contributions in agent_contributions.items():
18        for contrib in contributions:
19            subtask = shared_context.get_subtask(contrib.subtask_id)
20            contributions_text.append(
21                f"### {agent_id} 的分析\n"
22                f"任务: {subtask.type}\n"
23                f"结果:\n{contrib.result['answer']}\n"
24            )
25
26    prompt = f"""你是医疗助手的结果汇总专家。
27
28    用户问题: {question}
29
30    各专业Agent的分析:
31    {chr(10).join(contributions_text)}
32
33    请综合以上分析，生成一份完整、结构化的回答。包含：
34    1. 【综合分析】：整合各Agent的发现
35    2. 【核心建议】：可操作的具体建议
36    3. 【风险提示】：需要注意的风险点
37    """
38
39    response = await self.llm_client.chat_with_tools(
40        messages=[{"role": "user", "content": prompt}],
41        tools=None,
42        temperature=0.5
43    )
44
45    return response.content

```

Agent Loop

BaseAgent 基类

- 代码路径: `agents/base_agent.py`
- 职责: 提供 Agent 的通用能力(工具注册、执行、Swarm协作)

每个 Agent 有三个核心组件:

1. **LLMClient**: 负责与 LLM 交互
2. **AgentLoop**: 负责工具调用的循环逻辑
3. **ToolRegistry**: 存储该 Agent 可用的工具

代码块

```
1 class BaseAgent(ABC):
2     def __init__(
3         self,
4         agent_id: str,
5         config: Dict[str, Any],
6         llm_client: Optional[LLMClient] = None
7     ):
8         self.agent_id = agent_id
9         self.config = config
10        self.llm_client = llm_client or LLMClient()
11        self.loop = AgentLoop(max_iterations=10)
12
13        # 工具注册表
14        self.tool_registry = ToolRegistry()
15        self.register_tools()
16
17        # Swarm 协作相关
18        self.capabilities: List[str] = []
19        self.shared_context: Optional[Any] = None
20        self.identity_manager: Optional[Any] = None
```

工具执行统一通过 `execute_tool()` 方法, 从 `tool_registry` 中查找工具, 调用工具的 `function` 并传入参数

代码块

```
1 async def execute_tool(self, tool_name: str, arguments: Dict[str, Any]):
2     """执行工具调用"""
3     tool = self.tool_registry.get(tool_name)
4
5     if not tool:
6         return {"success": False, "error": f"Tool not found: {tool_name}"}
7
8     try:
9         logger.debug(f"Executing tool {tool_name} with args: {arguments}")
```

```

10         result = await tool.function(**arguments)
11         return result
12     except Exception as e:
13         return {
14             "success": False,
15             "error": f"Tool execution failed: {str(e)}"
16         }

```

下面实现了 Swarm 的核心逻辑:

- **上下文附加:** `attach_shared_context()` 让 Agent 可以访问共享上下文
- **任务处理:** `process_subtask()` 调用 Agent Loop 处理子任务

代码块

```

1  def attach_shared_context(self, shared_context: Any):
2      """附加 SharedContext (由 Swarm 调用) """
3      self.shared_context = shared_context
4
5  def attach_identity_manager(self, identity_manager: Any):
6      """附加 AgentIdentityManager (由 Swarm 调用) """
7      self.identity_manager = identity_manager
8
9  async def process_subtask(self, subtask: Any) -> Dict[str, Any]:
10      """
11      处理子任务 (Swarm 模式)
12
13      子类可以重写以实现自定义逻辑
14      默认实现: 运行 Agent Loop
15      """
16      # 使用 subtask.description 作为输入
17      input_data = {
18          'question': subtask.description,
19          'subtask_id': subtask.id,
20          'subtask_type': subtask.type
21      }
22
23      return await self.run_loop(input_data)

```

AgentLoop

- 代码路径: `core/agent_loop.py`
- 职责: LLM 驱动的 Think-Act-Observe 工具调用循环 (手动实现了 React 范式)

加入短期记忆 (最近10轮对话的历史上下文) 到上下文中。

```

1  messages = self._initialize_messages(agent, input_data, session_id)
2
3  # 记录用户消息到短期记忆
4  if self.short_term_memory and session_id:
5      user_message = messages[-1]["content"] if messages else str(input_data)
6      self.short_term_memory.add_message(
7          session_id=session_id,
8          role="user",
9          content=user_message
10     )
11 def _initialize_messages(self, agent, input_data, session_id):
12     """初始化消息列表，包含历史对话上下文"""
13     messages = []
14
15     # 系统提示词
16     system_prompt = agent.get_system_prompt()
17     if system_prompt:
18         messages.append({'role': 'system', 'content': system_prompt})
19
20     # 加载历史对话（短期记忆）
21     if self.short_term_memory and session_id:
22         history = self.short_term_memory.get_history(session_id, limit=10)
23         if history:
24             logger.info(f"Loaded {len(history)} historical messages")
25             messages.extend(history)
26
27     # 用户输入
28     user_message = agent.format_user_input(input_data)
29     messages.append({'role': 'user', 'content': user_message})
30
31     return messages

```

主循环: LLM → Tool Calls → Tool Results → LLM

- **Think:** LLM 分析当前状态，决定调用哪些工具
- **Act:** 执行工具调用，获取信息
- **Observe:** 将工具结果反馈给 LLM，继续思考

代码块

```

1  while state.should_continue():
2      state.iteration += 1
3
4      # 调用 LLM (可能返回 tool_calls)
5      llm_response: LLMResponse = await agent.llm_client.chat_with_tools(
6          messages=messages,

```

```

7         tools=tools_openai_format,
8         tool_choice="auto",
9         temperature=0.7
10    )
11
12    # 情况1: LLM 返回 tool_calls, 执行工具
13    if llm_response.has_tool_calls():
14        # 硬性限制: 检查是否已达到最大工具调用次数
15        if self.tool_call_count >= self.max_tool_calls:
16            logger.warning(f"已达到最大工具调用次数限制 ({self.max_tool_calls})")
17            # 强制要求 LLM 提供最终答案
18            messages.append({
19                'role': 'user',
20                'content': f'已完成 {self.max_tool_calls} 次信息检索。请基于已获取
21                的信息提供最终答复。'
22            })
23            continue
24
25    # 添加 assistant 消息 (包含 tool_calls)
26
27    messages.append(self._create_assistant_message_with_tools(llm_response))
28
29    # 记录 assistant 消息到短期记忆
30    if self.short_term_memory and session_id:
31        tool_names = [tc.name for tc in llm_response.tool_calls]
32        self.short_term_memory.add_message(
33            session_id=session_id,
34            role="assistant",
35            content=f"调用工具: {' '.join(tool_names)}"
36        )
37
38    # 执行每个工具调用
39    for tool_call in llm_response.tool_calls:
40        self.tool_call_count += 1
41
42    # Harness: 验证工具调用
43    if self.validator:
44        validation_result = self.validator.validate_tool_call(
45            agent.agent_id, tool_call.name
46        )
47        if not validation_result.get("valid"):
48            logger.warning(f"⚠️ 约束警告:
49            {validation_result.get('reason')}")
50
51    # 执行工具
52    tool_result = await agent.execute_tool(
53        tool_name=tool_call.name,

```

```

51         arguments=tool_call.arguments
52     )
53
54     # 添加工具结果消息
55     messages.append(
56         agent.llm_client.create_tool_message(
57             tool_call_id=tool_call.id,
58             tool_name=tool_call.name,
59             result=tool_result
60         )
61     )
62
63     # 记录工具结果到短期记忆
64     if self.short_term_memory and session_id:
65         result_summary = str(tool_result)[:200]
66         self.short_term_memory.add_message(
67             session_id=session_id,
68             role="tool",
69             content=f"{tool_call.name}: {result_summary}"
70         )
71
72     continue # 继续下一轮，让 LLM 分析工具结果
73
74     # 情况2: LLM 返回文本响应，任务完成
75     else:
76         final_answer = llm_response.content
77
78         # Harness: 验证和修复输出
79         if self.validator and final_answer:
80             validation_result = self.validator.validate_output(
81                 agent.agent_id, final_answer
82             )
83
84             if not validation_result.get("valid"):
85                 logger.warning(f"⚠️ 输出约束违规: {validation_result.get('violations')}")
86
87         # 自动修复
88         if self.auto_fixer and validation_result.get("auto_fixable"):
89             fixed_answer = self.auto_fixer.fix_output(
90                 final_answer,
91                 validation_result.get("auto_fixable", [])
92             )
93             if fixed_answer != final_answer:
94                 logger.info(f"🔧 输出已自动修复")
95                 final_answer = fixed_answer
96

```

```

97         # 记录最终回答到短期记忆
98         if self.short_term_memory and session_id:
99             self.short_term_memory.add_message(
100                 session_id=session_id,
101                 role="assistant",
102                 content=final_answer or "(empty response)"
103             )
104
105         result = {
106             'answer': final_answer,
107             'iterations': state.iteration,
108             'agent_id': agent.agent_id
109         }
110
111         state.mark_completed(result)
112         break

```

达到最大迭代次数时强制生成答案，这个降级机制确保 Agent 即使达到最大迭代次数也能给出答案，而不是直接失败

代码块

```

1  if not state.is_completed():
2      logger.warning(f"Max iterations reached without completion")
3
4      # 强制调用 LLM 生成最终总结（禁用工具调用）
5      messages.append({
6          'role': 'user',
7          'content': '请基于以上信息，提供最终的答复。注意：不要再调用工具。'
8      })
9
10     final_response = await agent.llm_client.chat_with_tools(
11         messages=messages,
12         tools=None, # 禁用工具
13         temperature=0.7
14     )
15
16     result = {
17         'answer': final_response.content or '抱歉，未能完成任务',
18         'iterations': state.iteration,
19         'warning': 'max_iterations_reached'
20     }
21
22     # 记录到短期记忆
23     if self.short_term_memory and session_id:
24         self.short_term_memory.add_message(

```



```

25         session_id=session_id,
26         role="assistant",
27         content=result['answer']
28     )
29
30     state.mark_completed(result)

```

Agent Swarm

SharedContext

- 代码路径: `swarm/shared_context.py`
- 职责: Agent 间的"信息素", 存储任务、贡献、事件

添加和查询子任务

代码块

```

1  def add_subtask(self, subtask: SubTask):
2      """添加子任务 (LeadAgent 调用) """
3      self.task_decomposition[subtask.id] = subtask
4
5      # 发布事件
6      self.publish_event(Event(
7          type=EventType.TASK_DECOMPOSED,
8          source_agent="lead_agent",
9          data={
10             "subtask_id": subtask.id,
11             "type": subtask.type,
12             "assigned_agent": subtask.assigned_agent
13         }
14     ))
15
16  def get_subtask(self, subtask_id: str) -> Optional[SubTask]:
17      """获取子任务"""
18      return self.task_decomposition.get(subtask_id)
19  def get_subtasks_for_agent(self, agent_id: str) -> List[SubTask]:
20      """
21      获取分配给指定 Agent 的待执行任务
22      """
23      tasks = []
24      for subtask in self.task_decomposition.values():
25          if subtask.assigned_agent == agent_id and subtask.can_be_executed():
26              tasks.append(subtask)
27
28      return tasksx

```

任务状态更新

代码块

```
1  def start_subtask(self, subtask_id: str) -> bool:
2      """
3      开始执行子任务
4
5      返回是否成功开始
6      """
7      subtask = self.get_subtask(subtask_id)
8      if not subtask or not subtask.can_be_executed():
9          return False
10
11     try:
12         subtask.start() # 更新为 IN_PROGRESS 状态
13
14         # 发布事件
15         self.publish_event(Event(
16             type=EventType.SUBTASK_STARTED,
17             source_agent=subtask.assigned_agent,
18             data={"subtask_id": subtask_id}
19         ))
20
21         return True
22     except ValueError:
23         return False
```

任务完成后记录贡献、LeadAgent 在汇总结果时获取所有 Agent 的贡献

代码块

```
1  def complete_subtask(
2      self,
3      subtask_id: str,
4      agent_id: str,
5      result: Dict[str, Any],
6      confidence: float = 1.0
7  ):
8      """完成子任务并添加贡献"""
9      subtask = self.get_subtask(subtask_id)
10     if not subtask:
11         raise ValueError(f"SubTask {subtask_id} not found")
12
13     if subtask.assigned_agent != agent_id:
14         raise ValueError(f"SubTask {subtask_id} not assigned to {agent_id}")
```

```

15
16     # 完成子任务
17     subtask.complete(result)
18
19     # 添加贡献
20     contribution = Contribution(
21         agent_id=agent_id,
22         subtask_id=subtask_id,
23         result=result,
24         confidence=confidence
25     )
26     self.agent_contributions[agent_id].append(contribution)
27
28     # 发布事件
29     self.publish_event(Event(
30         type=EventType.SUBTASK_COMPLETED,
31         source_agent=agent_id,
32         data={
33             "subtask_id": subtask_id,
34             "result_summary": str(result)[:200] # 简短摘要
35         }
36     ))
37 def get_contributions(
38     self,
39     agent_id: Optional[str] = None,
40     subtask_id: Optional[str] = None
41 ) -> List[Contribution]:
42     """
43     获取 Agent 贡献
44
45     可以根据 Agent ID 或 SubTask ID 过滤
46     """
47     if agent_id:
48         contributions = self.agent_contributions.get(agent_id, [])
49     else:
50         contributions = []
51         for agent_contribs in self.agent_contributions.values():
52             contributions.extend(agent_contribs)
53
54     if subtask_id:
55         contributions = [c for c in contributions if c.subtask_id ==
subtask_id]
56
57     return contributions

```



Workers 通过 SharedContext 实现**去中心化协作**:

- **不需要中心调度**: LeadAgent 只是发布任务，不控制执行
- **自主认领**: Workers 自己从 SharedContext 中查找并认领任务
- **并行执行**: 多个 Workers 可以同时处理不同的任务

Swarm执行流程

- 代码路径: `swarm/swarm_coordinator.py`
- 职责: Workers 自主认领任务并并行执行

`_process_with_swarm()` 方法启动 Swarm 协作:

代码块

```
1  async def _process_with_swarm(  
2      self,  
3      question: str,  
4      context: Dict[str, Any],  
5      assessment: Dict[str, Any],  
6      session_id: str,  
7      start_time: datetime  
8  ):   
9      # 创建 SharedContext  
10     shared_context = SharedContext(session_id=session_id)  
11  
12     # 附加 SharedContext 到所有 Worker  
13     for worker in self.worker_pool:  
14         worker.attach_shared_context(shared_context)  
15  
16     # 发布 Swarm 启动事件  
17     shared_context.publish_event(Event(  
18         type=EventType.SWARM_STARTED,  
19         source_agent="swarm_coordinator",  
20         data={"question": question, "num_subtasks":  
21             len(assessment["subtasks"])}  
22     ))  
23  
24     # Step 1: LeadAgent 分解任务并发布到 SharedContext  
25     subtasks = self.lead_agent.create_subtasks(assessment, shared_context)  
26  
27     # Step 2: Worker 执行分配的任务 (并行)  
28     tasks = []  
29     for worker in self.worker_pool:  
30         task = asyncio.create_task(  
31             self._worker_execute_assigned_tasks(worker, shared_context)
```

```

31         )
32         tasks.append(task)
33
34         # 等待所有 Worker 完成 (或超时)
35         try:
36             await asyncio.wait_for(
37                 asyncio.gather(*tasks, return_exceptions=True),
38                 timeout=90.0 # 超时时间 90 秒
39             )
40         except asyncio.TimeoutError:
41             logger.warning("Swarm execution timeout (90s)")
42
43         # Step 3: LeadAgent 汇总结果
44         final_answer = await self.lead_agent.synthesize_results(
45             question=question,
46             shared_context=shared_context,
47             timeout_occurred=timeout_occurred
48         )
49
50         # Step 4: 生成 SessionSummary
51         summary = SessionSummary.from_shared_context(
52             session_id=session_id,
53             question=question,
54             shared_context=shared_context,
55             final_answer=final_answer,
56             start_time=start_time,
57             end_time=end_time
58         )
59         self.session_manager.save_summary(summary)

```

`_worker_execute_assigned_tasks` : Worker 并行执行

代码块

```

1  async def _worker_execute_assigned_tasks(
2      self,
3      worker: Any,
4      shared_context: SharedContext
5  ):
6      """Worker 执行分配给它的任务"""
7      # 获取分配给该 Agent 的任务
8      assigned_tasks = shared_context.get_subtasks_for_agent(worker.agent_id)
9
10     if not assigned_tasks:
11         logger.debug(f'{worker.agent_id}: No assigned tasks')
12     return

```

```

13
14     # 【关键优化】并行执行所有分配的任务
15     tasks = []
16     for subtask in assigned_tasks:
17         logger.info(f"{worker.agent_id}: Starting {subtask.type}")
18         shared_context.start_subtask(subtask.id)
19
20         task = asyncio.create_task(
21             self._execute_single_subtask(worker, subtask, shared_context)
22         )
23         tasks.append(task)
24
25     # 等待所有任务完成
26     await asyncio.gather(*tasks, return_exceptions=True)
27
28     async def _execute_single_subtask(self, worker, subtask, shared_context):
29         """执行单个子任务"""
30         try:
31             result = await worker.process_subtask(subtask)
32             shared_context.complete_subtask(subtask.id, worker.agent_id, result)
33             logger.info(f"{worker.agent_id}: Completed {subtask.type}")
34         except Exception as e:
35             logger.error(f"{worker.agent_id}: Error in {subtask.type}: {e}")

```

记忆系统

短期记忆

- 代码路径: `memory/short_term.py`

默认获取最近10轮的对话加入到上下文，短期记忆有两种存储方式:

1. **内存存储** (默认): 无需配置，数据存储在 Python 字典中，进程重启后丢失
2. **Redis 存储** (可选): 需要配置 Redis，支持持久化和分布式访问

获取最近消息，添加了熵管理模块，对会话记录进行自动去重+压缩

代码块

```

1  def get_recent_messages(
2      self,
3      session_id: str,
4      limit: int = 50
5  ) -> List[Dict[str, str]]:
6      """
7      获取最近的消息（自动熵管理）
8

```

```

9      熵管理策略:
10     1. 估算熵等级 (>10条消息时), 警告高熵状态
11     2. 统一使用 auto_clean 进行去重和压缩
12     """
13     history = self.get_session(session_id)
14     if history:
15         messages = history.get_recent_messages(limit)
16
17         # Harness Engineering: 统一熵管理
18         if self.entropy_manager and len(messages) > 0:
19             # 估算熵 (用于监控系统健康)
20             if len(messages) > 10:
21                 entropy_info = self.entropy_manager.estimate_entropy(messages)
22                 if entropy_info["entropy_level"] == "high":
23                     logger.warning(
24                         f"

```

长期记忆

- 代码位置: `memory/long_term.py`
- 职责: 跨会话记忆, 根据向量相似度检索历史案例

长期记忆使用 Mem0 云服务存储会话总结:

代码块

```

1  class LongTermMemory:
2      """长期记忆 (Mem0 云服务) """
3
4      def __init__(self, user_id: str = "medix_default_user"):
5          self.user_id = user_id
6          self.enabled = False

```

```

7         self.client = None
8         self.entropy_manager = MemoryEntropyManager() if ENTROPY_ENABLED else
None
9
10        # 尝试初始化 Mem0 客户端
11        try:
12            from mem0 import MemoryClient
13
14            # 检查 Mem0 配置
15            if not MEM0_CONFIG.get("api_key"):
16                logger.warning("Mem0 API key not configured, long-term memory
disabled")
17                return
18
19            self.client = MemoryClient(api_key=MEM0_CONFIG["api_key"])
20            self.enabled = True
21            logger.info("✅ Long-term memory (Mem0) enabled")
22
23        except ImportError:
24            logger.warning("Mem0 not installed, long-term memory disabled")
25        except Exception as e:
26            logger.error(f"Failed to initialize Mem0: {e}")

```

检索相似会话（自动去重）

代码块

```

1  def search_similar_sessions(
2      self,
3      query: str,
4      limit: int = 5
5  ) -> List[Dict[str, Any]]:
6      """
7      搜索相似的历史会话（向量相似度搜索，自动去重）
8
9      Args:
10         query: 查询文本（通常是用户问题）
11         limit: 返回结果数量
12
13     Returns:
14         相似会话列表，每个包含 memory_id、content、score、metadata
15     """
16     if not self.enabled:
17         return []
18
19     try:

```



```

20         results = self.mem0.search(
21             query=query,
22             user_id="medix_user",
23             limit=limit * 2 # 多获取一些以便去重后有足够结果
24         )
25
26         # 添加调试日志
27         logger.debug(f"Mem0 search called with limit={limit * 2}")
28
29         # 格式化结果
30         formatted_results = []
31
32         # 处理不同格式的 Mem0 返回值
33         if isinstance(results, dict):
34             results_list = results.get("results", [])
35             logger.debug(f"Mem0 returned dict with {len(results_list)}
36 results (requested limit={limit * 2})")
37         elif isinstance(results, list):
38             results_list = results
39             logger.debug(f"Mem0 returned list with {len(results_list)}
40 results (requested limit={limit * 2})")
41         else:
42             results_list = []
43
44         for result in results_list:
45             formatted_results.append({
46                 "memory_id": result.get("id", "unknown"),
47                 "content": result.get("memory", result.get("text", "")),
48                 "score": result.get("score", 0.0),
49                 "metadata": result.get("metadata", {}),
50                 "timestamp": result.get("metadata", {}).get("timestamp")
51             })
52
53         # Harness Engineering: 熵管理 - 去重相似会话
54         if self.entropy_manager and len(formatted_results) > 0:
55             formatted_results =
56 self.entropy_manager.deduplicate_sessions(formatted_results)
57
58         # 限制返回数量
59         formatted_results = formatted_results[:limit]
60
61         logger.info(f"Found {len(formatted_results)} similar sessions for
62 query: {query[:50]}...")
63         return formatted_results
64
65     except Exception as e:

```

```
63         logger.error(f"Failed to search similar sessions: {e}")
64         return []
```



Mem0简要介绍

Mem0 是一款聚焦于LLM记忆能力增强的框架，核心可概括为以下几点：

1. **结构化记忆管理**：为 LLM 构建可持久化、可检索的记忆系统，将用户交互、上下文信息、领域知识等转化为结构化的记忆单元（本项目中用作长期记忆），替代 LLM 仅依赖上下文窗口的临时记忆方式。
 2. **智能记忆检索与融合**：通过RAG、语义相似度匹配等技术，在 LLM 生成响应时，精准调取与当前场景相关的历史记忆，让模型能记住长期交互中的关键信息（如用户偏好、历史对话结论、专属知识）。
 3. **记忆的动态演进**：支持记忆的更新、删除、优先级调整，模拟人类记忆的遗忘、强化逻辑，让记忆体系能随交互过程动态迭代，而非静态存储，适配真实场景中信息的变化。
 4. **轻量化与适配性**：旨在以低侵入的方式集成到现有 LLM 应用中，无需重构模型本身，通过外部记忆层为不同 LLM赋能，兼顾易用性和兼容性。
- **存储内容**：原始文本、向量、元数据。
 - **检索机制**：语义相似度+关键词匹配加权融合。
 - **记忆架构**：
 - **User Memory**: 用户级，跨会话持久化（本项目只使用user级）
 - **Session Memory**: 会话级，单次对话上下文
 - **Agent Memory**: Agent 级，Agent 自身学习到的知识和模式

记忆加入到上下文中

- 代码位置： `swarm/swarm_coordinator.py`

记忆系统在 SwarmCoordinator 的 `process()` 方法中被使用，上下文会传递给：

1. **LeadAgent**: 分解任务时参考历史案例
2. **Worker Agents**: 处理任务时利用对话历史

代码块

```
1 recent_history = self.short_term_memory.get_recent_messages(  
2     session_id=session_id,  
3     limit=10 # 最近5轮对话 (10条消息)  
4 )  
5  
6 # 2. 检索长期记忆 (相似历史会话)
```

```

7     similar_memories = self.long_term_memory.search_similar_sessions(
8         query=question,
9         limit=3
10    )
11
12    # 3. 构建增强上下文
13    enhanced_context = context or {}
14
15    # 添加短期记忆
16    if recent_history:
17        enhanced_context["recent_history"] = [
18            {"role": msg.get("role", ""), "content": msg.get("content", "")[:100]}
19            for msg in recent_history[-6:] # 最近3轮对话 (6条消息)
20        ]
21
22    # 添加长期记忆
23    if similar_memories:
24        enhanced_context["historical_cases"] = [
25            {"summary": mem["content"], "score": mem["score"]}
26            for mem in similar_memories
27        ]

```

Skills

- 代码路径: `core/skill_loader.py`

系统会扫描 `.claude/skills/` 目录, 自动发现所有符合规范的 Skills。每个 Skill 必须包含:

- `SKILL.md` (含 YAML frontmatter, 包含 name 和 description)
- `script/xxx.py` (含 Skill 主函数)

当前 9 个 Skills:

1. `search_knowledge` - 搜索医学知识库 (Milvus RAG)
2. `recommend_lifestyle` - 生活方式建议 (Milvus RAG)
3. `assess_risk` - 风险评估 (规则引擎 + Milvus RAG)
4. `analyze_symptoms` - 症状分析 (规则引擎 + Milvus RAG)
5. `disease_code` - ICD-10 编码查询 (Milvus RAG)
6. `clinical_guideline` - 临床指南检索 (Milvus RAG)
7. `deep_research` - 深度研究 (Web Search + Milvus RAG + 证据综合)
8. `search_history` - 搜索当前会话历史 (短期记忆)
9. `search_similar_cases` - 搜索相似历史案例 (长期记忆 Mem0)

扫描目录，自动发现并加载 Skills

代码块

```
1  def discover_skills(project_root: Path = None) -> List[Dict]:
2      """
3      自动扫描 .claude/skills 目录，发现所有 skills
4
5      Returns:
6          [
7              {
8                  "name": "search-knowledge",
9                  "function_name": "search_knowledge",
10                 "script_name": "search",
11                 "metadata": { "name": "...", "description": "..." },
12                 "function": <function>
13             },
14             ...
15         ]
16     """
17     if project_root is None:
18         project_root = Path(__file__).parent.parent
19
20     skills_dir = project_root / ".claude" / "skills"
21     discovered_skills = []
22
23     # 遍历所有子目录
24     for skill_dir in skills_dir.iterdir():
25         if not skill_dir.is_dir():
26             continue
27
28         skill_name = skill_dir.name
29
30         # 1. 查找 SKILL.md 并解析 frontmatter
31         skill_md_path = skill_dir / "SKILL.md"
32         if not skill_md_path.exists():
33             logger.debug(f"Skipping {skill_name}: no SKILL.md found")
34             continue
35
36         metadata = parse_skill_md(skill_md_path)
37         if not metadata:
38             logger.warning(f"Skipping {skill_name}: failed to parse SKILL.md")
39             continue
40
41         # 2. 查找 script 目录中的 Python 文件
42         script_dir = skill_dir / "script"
```

```

43     script_files = [f for f in script_dir.iterdir() if f.suffix == '.py'
44 and f.name != '__init__.py']
45
46     if not script_files:
47         logger.warning(f"Skipping {skill_name}: no Python script found")
48         continue
49
50     script_file = script_files[0]
51     script_name = script_file.stem
52
53     # 3. 推断函数名 (kebab-case → snake_case)
54     function_name = skill_name.replace('-', '_')
55
56     # 4. 动态加载函数并验证
57     try:
58         func = load_skill_function(skill_name, script_name, function_name,
59 project_root)
60         discovered_skills.append({
61             "name": skill_name,
62             "function_name": function_name,
63             "script_name": script_name,
64             "metadata": metadata,
65             "function": func
66         })
67         logger.info(f"✅ Discovered skill: {skill_name}")
68     except (FileNotFoundError, AttributeError) as e:
69         logger.warning(f"⚠️ Skipping {skill_name}: {e}")
70
71     logger.info(f"Discovered {len(discovered_skills)} skills in total")
72     return discovered_skills

```

解析 YAML frontmatter

代码块

```

1  def parse_skill_md(file_path: Path) -> Optional[Dict]:
2      """
3      解析 SKILL.md 文件的 YAML frontmatter
4
5      Example frontmatter:
6      ---
7      name: search-knowledge
8      description: Search medical knowledge base.
9      ---
10     """
11     try:

```

```

12     with open(file_path, 'r', encoding='utf-8') as f:
13         content = f.read()
14
15         # 解析 YAML frontmatter
16         if content.startswith('---'):
17             end_idx = content.find('---', 3)
18             if end_idx != -1:
19                 yaml_content = content[3:end_idx]
20                 data = yaml.safe_load(yaml_content)
21                 return data
22         return None
23     except Exception as e:
24         logger.warning(f"Error parsing {file_path}: {e}")
25         return None

```

skill注册、调用和转化成function calling格式，加入上下文（`core/skill_registry.py`）

代码块

```

1  class SkillRegistry:
2      """Skill 注册表，直接管理 Skill 函数并提供 OpenAI 格式转换"""
3
4      def __init__(self):
5          self.skills: Dict[str, Dict[str, Any]] = {}
6
7      def register(self, name: str, function: Callable, description: str,
8                  parameters: List[SkillParameter]):
9          """注册 skill (无需 Tool 包装)"""
10         self.skills[name] = {
11             'function': function,
12             'description': description,
13             'parameters': parameters,
14             'is_async': inspect.iscoroutinefunction(function)
15         }
16
17         async def execute(self, name: str, **kwargs) -> Dict[str, Any]:
18             """执行 Skill"""
19             skill = self.skills.get(name)
20             if skill['is_async']:
21                 result = await skill['function'](**kwargs)
22             else:
23                 loop = asyncio.get_event_loop()
24                 result = await loop.run_in_executor(None, lambda: skill['function']
25                 (**kwargs))
26             return result

```

```

26     def to_openai_format(self) -> List[Dict[str, Any]]:
27         """直接转换为 OpenAI function calling 格式"""
28         tools = []
29         for name, skill in self.skills.items():
30             properties = {}
31             required = []
32             for param in skill['parameters']:
33                 properties[param.name] = {
34                     'type': param.type,
35                     'description': param.description
36                 }
37             if param.required:
38                 required.append(param.name)
39
40         tools.append({
41             'type': 'function',
42             'function': {
43                 'name': name,
44                 'description': skill['description'],
45                 'parameters': {
46                     'type': 'object',
47                     'properties': properties,
48                     'required': required
49                 }
50             }
51         })
52     return tools

```

agent注册时，自动注册发现的 Skills (agents/skill_registry_mixin.py)

代码块

```

1     class SkillRegistryMixin:
2         """统一注册所有 Skills (自动发现模式) """
3
4     def register_all_skills(self):
5         """自动扫描并注册所有 Skills"""
6         project_root = Path(__file__).parent.parent
7         discovered = discover_skills(project_root)
8
9         # 自动注册所有发现的 skills
10        for skill_info in discovered:
11            function_name = skill_info["function_name"]
12            metadata = skill_info["metadata"]
13            func = skill_info["function"]
14

```

```

15         # 从 metadata 获取描述
16         description = metadata.get("description", f"Skill:
{skill_info['name']}")
17
18         # 根据函数签名自动推断参数
19         parameters = self._infer_skill_parameters(skill_info)
20
21         # 直接注册到 SkillRegistry (无 Tool 中间层)
22         self.skill_registry.register(
23             name=function_name,
24             function=func,
25             description=description,
26             parameters=parameters
27         )
28         logger.info(f"✅ Registered skill: {function_name}")
29
30         logger.info(f"Total {len(discovered)} skills registered")
31
32     def _infer_skill_parameters(self, skill_info: dict) -> list:
33         """
34         从函数签名自动推断工具参数
35
36         推断规则:
37         - 包含 count, limit, max, iterations → number
38         - 其他 → string
39         - 有默认值 → optional
40         - 无默认值 → required
41         """
42         import inspect
43
44         func = skill_info["function"]
45         sig = inspect.signature(func)
46         parameters = []
47
48         for param_name, param in sig.parameters.items():
49             if param_name in ["self", "args", "kwargs"]:
50                 continue
51
52             # 判断是否必需
53             required = param.default == inspect.Parameter.empty
54
55             # 推断类型
56             param_type = "string"
57             if any(kw in param_name for kw in ["count", "limit", "max",
"iterations"]):
58                 param_type = "number"
59

```



```

60         # 生成描述
61         param_desc = param_name.replace('_', ' ').title()
62
63         parameters.append(ToolParameter(
64             param_name,
65             param_type,
66             param_desc,
67             required
68         ))
69
70     return parameters

```

举一个具体的skills的例子： `.claude/skills/search-knowledge/script/search.py`，检索医学知识库

代码块

```

1  async def search_knowledge(query: str, max_results: int = 5) -> Dict[str, Any]:
2      """搜索医学知识库 (Milvus RAG) """
3      kb = get_knowledge_base() # 获取知识库单例 (避免重复加载模型)
4
5      # 使用 Milvus 进行语义检索
6      results = kb.search(query=query, top_k=max_results, filter_type=None)
7
8      # 格式化结果
9      formatted_results = []
10     for doc in results:
11         formatted_results.append({
12             "title": f"关于{doc['metadata'].get('disease', query)}的医学信息",
13             "content": doc["content"],
14             "score": doc["score"]
15         })
16
17     return {
18         "answer": format_results(formatted_results),
19         "total_found": len(formatted_results),
20         "query": query
21     }

```

再看下deepresearch 这个skill 的实现 (`.claude/skills/deep-research/script/research.py`)

融合了Web Search + Milvus RAG，可以并行检索，并对检索到的证据进行汇总。

代码块

```

1  async def deep_research(query: str, max_iterations: int = 2) -> Dict[str, Any]:
2      """
3      深度研究
4
5      Args:
6          query: 研究问题
7          max_iterations: 最大迭代次数（默认2）
8
9      Returns:
10         {
11             "answer": "格式化的研究报告",
12             "findings": ["发现1", "发现2"],
13             "confidence": "high/medium/low"
14         }
15     """
16     # 1. 创建工作流
17     workflow = DeepResearchWorkflow()
18
19     # 2. 执行研究
20     report = await workflow.run(
21         question=query,
22         max_web_results=max_iterations * 5, # 动态控制搜索深度
23         max_kb_results=max_iterations * 3
24     )
25
26     # 3. 返回格式化结果
27     return {
28         "answer": format_research_report(query, report),
29         "findings": report.key_findings, # List[str]
30         "confidence": "high" if report.confidence > 0.7 else "medium" if
report.confidence > 0.4 else "low",
31         "sources": len(report.sources),
32         "evidence_level": report.evidence_level, # A/B/C
33         "status": "completed",
34         "data_sources": "Web Search + Milvus RAG + Evidence Synthesis"
35     }

```

代码块

```

1  async def run(
2      self,
3      question: str,
4      max_web_results: int = 10,
5      max_kb_results: int = 5
6  ) -> ResearchReport:
7      """

```

```
8         执行深度研究
9
10        Args:
11            question: 研究问题
12            max_web_results: 最大网络搜索结果数
13            max_kb_results: 最大知识库检索结果数
14
15        Returns:
16            研究报告
17        """
18        logger.info(f"Starting DeepResearch for: {question}")
19
20        # Step 1: 查询规划
21        sub_queries = await self._plan_queries(question)
22        logger.info(f"Planned {len(sub_queries)} sub-queries")
23
24        # Step 2: 并行搜索
25        web_results: List[SearchResult] = []
26        kb_results: List[Document] = []
27
28        search_tasks = []
29
30        if self.use_web_search and self.web_search:
31            # 网络搜索
32            for query in sub_queries[:3]: # 限制子查询数量
33                search_tasks.append(
34                    self.web_search.search(query, max_results=max_web_results
35// len(sub_queries))
36                )
37
38        if self.use_knowledge_base and self.knowledge_base:
39            # 从 Milvus 知识库检索
40            for query in sub_queries[:3]:
41                search_tasks.append(
42                    self._search_milvus(query, top_k=max_kb_results //
43len(sub_queries))
44                )
45
46        # 并行执行
47        if search_tasks:
48            results = await asyncio.gather(*search_tasks,
49return_exceptions=True)
50
51        # 分离结果
52        for result in results:
53            if isinstance(result, Exception):
54                logger.warning(f"Search task failed: {result}")
```

```

52             continue
53
54         if isinstance(result, list):
55             if len(result) > 0:
56                 if isinstance(result[0], SearchResult):
57                     web_results.extend(result)
58                 elif isinstance(result[0], dict):
59                     # Milvus 返回的是字典列表
60                     kb_results.extend(result)
61
62     logger.info(f"Collected {len(web_results)} web results,
63 {len(kb_results)} KB results")
64
65     # Step 3: 证据综合
66     report = await self.synthesizer.synthesize(
67         query=question,
68         web_results=web_results,
69         kb_results=kb_results
70     )
71     if not report.key_findings:
72         logger.warning("Report has no key findings")
73
74     if not report.summary:
75         logger.warning("Report has no summary")
76
77     logger.info("DeepResearch completed")
78     return report

```

接下来看一下RAG知识库的实现，采用Milvus知识库。 `knowledge/milvus_kb.py`

代码块

```

1  def _chunk_text(self, text: str, chunk_size: int = 1024, overlap: int = 100) -
    > List[str]:
2      """
3      分块文本（支持重叠）
4
5      Args:
6          text: 原始文本
7          chunk_size: 块大小（字符数）
8          overlap: 重叠字符数（避免语义断裂）
9
10     Returns:
11         文本块列表
12     """

```

```

13     if len(text) <= chunk_size:
14         return [text]
15
16     chunks = []
17     start = 0
18     while start < len(text):
19         end = start + chunk_size
20         chunk = text[start:end]
21         chunks.append(chunk)
22         start = end - overlap # 重叠
23
24     return chunks
25
26 def add_documents(self, documents: List[Dict[str, Any]], chunk_size: int =
1024) -> int:
27     """
28     添加文档到知识库（支持分块）
29
30     Args:
31         documents: 文档列表，每个文档包含 id, content, metadata
32         chunk_size: 分块大小（字符数），默认 1024
33
34     Returns:
35         成功添加的文档块数量
36     """
37     if not documents:
38         logger.warning("No documents to add")
39         return 0
40
41     logger.info(f"Adding {len(documents)} documents to knowledge base
(chunk_size={chunk_size})...")
42
43     # Step 1: 分块
44     all_chunks = []
45     for doc in documents:
46         chunks = self._chunk_text(doc["content"], chunk_size=chunk_size)
47         for i, chunk in enumerate(chunks):
48             metadata = doc.get("metadata", {}).copy()
49             metadata["doc_id"] = doc["id"]
50             metadata["chunk_id"] = i
51             metadata["total_chunks"] = len(chunks)
52
53             all_chunks.append({
54                 "content": chunk,
55                 "metadata": metadata
56             })
57

```

```

58     logger.info(f"Split into {len(all_chunks)} chunks")
59
60     # Step 2: 向量化
61     contents = [chunk["content"] for chunk in all_chunks]
62     vectors = self.embedding_model.encode(contents, show_progress_bar=True)
63
64     # Step 3: 准备数据
65     data = []
66     for i, chunk in enumerate(all_chunks):
67         data.append({
68             "vector": vectors[i].tolist(),
69             "content": chunk["content"],
70             "metadata": json.dumps(chunk["metadata"], ensure_ascii=False)
71         })
72
73     # Step 4: 插入到 Milvus
74     self.milvus_client.insert(self.collection_name, data)
75     logger.info(f"Successfully added {len(data)} chunks")
76
77     return len(data)
78
79 def search(
80     self,
81     query: str,
82     top_k: int = 5,
83     filter_type: Optional[str] = None
84 ) -> List[Dict[str, Any]]:
85     """
86     检索相关文档
87
88     Args:
89         query: 查询文本
90         top_k: 返回top K个结果
91         filter_type: 可选的类型过滤 (如 "lifestyle", "disease_classification")
92
93     Returns:
94         文档列表, 每个文档包含 id, content, metadata, score
95     """
96     logger.debug(f"Searching for: {query} (top_k={top_k}, filter_type={filter_type})")
97
98     # Step 1: 向量化查询
99     query_vector = self.embedding_model.encode([query])[0]
100
101     # Step 2: 构建过滤条件 (JSON 字符串匹配)
102     filter_expr = None
103     if filter_type:

```

```

104         filter_expr = f'metadata like "%\\"type\\": \\"{filter_type}\\\\""'
105
106     # Step 3: 执行检索
107     try:
108         results = self.milvus_client.search(
109             collection_name=self.collection_name,
110             data=[query_vector.tolist()],
111             limit=top_k,
112             filter=filter_expr,
113             output_fields=["content", "metadata"]
114         )
115     except Exception as e:
116         logger.error(f"Search failed: {e}")
117         return []
118
119     # Step 4: 格式化结果
120     documents = []
121     for hits in results:
122         for hit in hits:
123             try:
124                 documents.append({
125                     "id": hit["id"],
126                     "content": hit["entity"]["content"],
127                     "metadata": json.loads(hit["entity"]["metadata"]),
128                     "score": 1 - hit["distance"] # COSINE: 转换为相似度分数
129                 })
130             except Exception as e:
131                 logger.warning(f"Failed to parse result: {e}")
132                 continue
133
134     logger.debug(f"Found {len(documents)} documents")
135     return documents

```

Harness

[harness engineering](#)是对模型功能的约束，其**核心理念**是人类设计约束，AI 代理执行，让 AI 在明确约束下自主工作、自我修正。

本项目 中实现了以下约束，感兴趣的同学可以再做拓展：

Skill 调用验证

- 代码路径： `constraints/validator.py`

`severity: "warning"` = 记录警告但不阻止执行（软约束）。允许 Agent 在需要时跨领域调用 Skills，提高灵活性。

```

1  def validate_tool_call(self, agent_id: str, tool_name: str) -> Dict[str, Any]:
2      """
3      验证 Skill 调用是否允许
4
5      Args:
6          agent_id: Agent ID
7          tool_name: Skill 名称
8
9      Returns:
10         {"valid": bool, "reason": str, "severity": str}
11     """
12     agent_constraints = self.agent_constraints['agents'].get(agent_id, {})
13     allowed_tools = agent_constraints.get('allowed_tools', [])
14
15     # 如果 allowed_tools 为空, 表示没有限制
16     if not allowed_tools:
17         return {"valid": True}
18
19     # 检查 Skill 是否在允许列表中
20     if tool_name not in allowed_tools:
21         reason = f"工具 {tool_name} 不在 {agent_id} 的推荐工具列表中"
22         logger.warning(f"⚠️ {reason}")
23         return {
24             "valid": False,
25             "reason": reason,
26             "severity": "warning" # 警告级别 (不阻止执行, 只记录)
27         }
28
29     return {"valid": True}

```

输出内容验证并修复

- 代码位置: `constraints/validator.py`

代码块

```

1  def validate_output(self, agent_id: str, output: str) -> Dict[str, Any]:
2      """
3      验证输出是否符合约束
4
5      Returns:
6          {
7              "valid": bool,
8              "violations": List[str], # 违规列表
9              "auto_fixable": List[str] # 可自动修复的违规
10         }
11     """

```



```
12 agent_constraints = self.agent_constraints['agents'].get(agent_id, {})
13 output_constraints = agent_constraints.get('output_constraints', [])
14
15 violations = []
16 auto_fixable = []
17
18 # 1. 检查免责声明
19 if 'must_include_disclaimer' in output_constraints:
20     if '免责声明' not in output and '仅供参考' not in output:
21         violations.append("缺少免责声明")
22         auto_fixable.append("add_disclaimer")
23
24 # 2. 检查长度限制
25 max_length_constraint = next(
26     (c for c in output_constraints if isinstance(c, dict) and
27      'max_response_length' in c),
28     None
29 )
30 if max_length_constraint:
31     max_length = max_length_constraint['max_response_length']
32     if len(output) > max_length:
33         violations.append(f"回答过长 ({len(output)} > {max_length}字) ")
34
35 # 3. 检查高危症状必须建议就医
36 if 'must_recommend_doctor_visit_if_high_risk' in output_constraints:
37     high_risk_keywords = ["胸痛", "呼吸困难", "昏厥", "剧烈头痛", "心悸", "突然
38     视力模糊"]
39     if any(kw in output for kw in high_risk_keywords):
40         if "就医" not in output and "急诊" not in output and "医院" not in
41         output:
42             violations.append("高危症状未建议就医")
43             auto_fixable.append("add_emergency_warning")
44
45 # 4. 检查禁止行为：明确诊断
46 forbidden_actions = agent_constraints.get('forbidden_actions', [])
47 if 'diagnose_disease' in forbidden_actions:
48     if any(phrase in output for phrase in ["您患有", "确诊为", "肯定是"]):
49         violations.append("包含明确诊断（越界行为）")
50
51 # 5. 检查禁止行为：药物处方（精细检测，避免误报）
52 if 'prescribe_medication' in forbidden_actions:
53     import re
54     # 只检测具体剂量的处方模式
55     if re.search(r'（药物|药品）.{0,10}(\d+\s*(mg|g|毫克|克))', output):
56         violations.append("包含具体药物处方（越界行为）")
57     elif re.search(r'每(日|天|次)}.{0,5}\d+\s*次.{0,10}(\d+\s*(mg|g|毫克|
58     克))', output):
```

```

55         violations.append("包含具体药物处方（越界行为）")
56
57     return {
58         "valid": len(violations) == 0,
59         "violations": violations,
60         "auto_fixable": auto_fixable
61     }

```

- 代码位置: `validation/auto_fixer.py`

代码块

```

1  class AutoFixer:
2      """自动修复器"""
3
4      def fix_output(self, output: str, auto_fixable: List[str]) -> str:
5          """
6              自动修复输出
7
8              Args:
9                  output: 原始输出
10                 auto_fixable: 可修复的违规列表（来自 ConstraintValidator）
11
12             Returns:
13                 修复后的输出
14             """
15             fixed_output = output
16
17             for fix_type in auto_fixable:
18                 if fix_type == "add_disclaimer":
19                     fixed_output = self.fix_missing_disclaimer(fixed_output)
20                 elif fix_type == "add_emergency_warning":
21                     fixed_output = self.fix_high_risk_warning(fixed_output)
22
23             if fixed_output != output:
24                 logger.info("🔧 输出已自动修复")
25
26             return fixed_output
27
28     def fix_missing_disclaimer(self, output: str) -> str:
29         """自动添加免责声明"""
30         if "免责" not in output and "仅供参考" not in output:
31             disclaimer = "\n\n【免责声明】\n以上信息仅供参考，不能替代专业医生的诊断和
治疗。如有疑问，请及时就医。"
32             logger.debug("+ 自动添加免责声明")
33             return output + disclaimer

```

```

34         return output
35
36     def fix_high_risk_warning(self, output: str) -> str:
37         """自动添加高危症状警告"""
38         high_risk_keywords = ["胸痛", "呼吸困难", "昏厥", "剧烈头痛", "心悸", "突然
视力模糊"]
39
40         if any(kw in output for kw in high_risk_keywords):
41             if "就医" not in output and "急诊" not in output:
42                 warning = "⚠️ **重要提醒**：您描述的症状可能提示严重问题，建议立即就医
或拨打急救电话120，不要延误。\\n\\n"
43                 logger.debug("+ 自动添加高危症状警告")
44                 return warning + output
45
46         return output

```

熵管理

- 代码位置： `memory/entropy_manager.py`

清理冗余信息，保持系统简洁

代码块

```

1  class EntropyManager:
2      """记忆熵管理器 - 系统复杂度的'垃圾回收'"""
3
4      def deduplicate_messages(self, messages: List[Dict]) -> List[Dict]:
5          """去除完全重复的消息（基于 MD5 哈希）"""
6          unique_messages = []
7          seen_hashes = {}
8
9          for msg in messages:
10             role = msg.get("role", "")
11             content = msg.get("content", "")
12             content_hash = hashlib.md5(f"{role}:{
{content}}".encode()).hexdigest()
13
14             if content_hash not in seen_hashes:
15                 unique_messages.append(msg)
16                 seen_hashes[content_hash] = msg
17             else:
18                 logger.debug(f"Duplicate message removed: {content[:50]}...")
19
20         return unique_messages
21
22     def compress_session_history(

```

```

23         self,
24         messages: List[Dict],
25         max_messages: int = 10
26     ) -> List[Dict]:
27         """压缩会话历史（保留最近消息，压缩早期消息为摘要）"""
28         if len(messages) <= max_messages:
29             return messages
30
31         # 保留最近的消息
32         recent = messages[-max_messages:]
33
34         # 压缩更早的消息
35         older = messages[:-max_messages]
36         compressed = self._compress_older_messages(older)
37
38         return compressed + recent
39
40     def _compress_older_messages(self, messages: List[Dict]) -> List[Dict]:
41         """将早期消息压缩为摘要"""
42         if not messages:
43             return []
44
45         # 简单实现：提取关键信息
46         user_questions = [m["content"] for m in messages if m["role"] ==
47 "user"]
48
49         summary = {
50             "role": "system",
51             "content": f"[早期对话摘要] 用户曾询问：{''.join(user_questions[:3])}..."
52         }
53
54         return [summary]
55
56     def estimate_entropy(self, messages: List[Dict]) -> Dict[str, Any]:
57         """估算系统熵等级并提供优化建议"""
58         total = len(messages)
59
60         # 计算重复率
61         unique = len(self.deduplicate_messages(messages))
62         duplicate_rate = (total - unique) / total if total > 0 else 0
63
64         # 评估熵等级
65         if total < 10:
66             entropy_level = "low"
67         elif total < 50:
68             entropy_level = "medium"

```

```
68         else:
69             entropy_level = "high"
70
71         # 生成建议
72         recommendations = []
73         if total > 50:
74             recommendations.append("建议压缩历史消息（当前 > 50 条）")
75         if duplicate_rate > 0.2:
76             recommendations.append(f"检测到 {duplicate_rate*100:.1f}% 重复消息，
建议去重")
77
78         return {
79             "total_messages": total,
80             "unique_messages": unique,
81             "duplicate_rate": duplicate_rate,
82             "entropy_level": entropy_level,
83             "recommendations": recommendations
84         }
```